

AUDIT TECHNIQUE & SÉCURITÉ

DOCUMENTATION ELSON

Document d'audit technique — établi par lecture exhaustive du dépôt local `elson-main` (aucune exécution, aucun déploiement, aucune modification de l'application)

Architecture & Sécurité

Règle appliquée : seul ce qui est réellement présent dans le dépôt est documenté. Aucune fonctionnalité, aucun fichier, aucune configuration n'a été inventé. Aucun secret n'est affiché (les valeurs sensibles sont désignées par leur nom de variable uniquement).

Analyse statique exhaustive du dépôt `elson-main` : architecture, API,

Convention de statut, configuration, risques de sécurité (OWASP) et

prérequis de déploiement sur un socle GCP sécurisé.

STATUT	SIGNIFICATION
✓ Vérifié	Constaté directement dans un fichier du dépôt (chemin cité).
🔴 À confirmer	Dépend de la configuration runtime / du serveur / de <code>.env</code> , non observable dans le dépôt.
🔴 Absent	Rien dans le dépôt ne couvre ce point.
💡 Recommandé	Proposition d'amélioration, non présente aujourd'hui.

Périmètre analysé : racine du dépôt, `src/` (frontend), `backend/src/` (API), `backend/sql/` (74 fichiers SQL), `deploy/`, `docs/`, `public/`, `docker-compose.yml`, `Caddyfile`, `Dockerfile`, `backend/Dockerfile`. **Volumétrie** : ~14 200 lignes TypeScript backend, ~20 400 lignes TypeScript/TSX frontend, 225 endpoints HTTP, 54 tables PostgreSQL définies, 17 vues, 15 fonctions/triggers.

1. Présentation, objectifs et périmètre

1.1 Nature du produit

✓ **Vérifié** — `README.md:1-3`

Elson est une plateforme de *crowdsourcing gamifiée* destinée à construire des jeux de données parallèles **texte + audio** pour les langues nationales de Mauritanie, en commençant par l'**arabe hassaniya**. Les contributeurs traduisent des phrases sources (EN/FR/AR) vers le hassaniya en écriture arabe, **enregistrent leur voix**, puis évaluent le travail des autres. Les données de meilleure qualité alimentent des modèles **ASR** (reconnaissance vocale), **MT** (traduction automatique) et **TTS** (synthèse vocale).

Initiative portée par **ADST** (African Digital Services & Technologies) en partenariat avec **RIM AI** (`README.md:3` , `backend/src/services/email.ts:56`).

1.2 Objectifs fonctionnels observés dans le code

OBJECTIF	PREUVE DANS LE DÉPÔT	STATUT
Collecter des traductions hassaniya + audio	<code>backend/src/routes/phrases.ts</code> (<code>POST /submit</code>)	✓ Vérifié
Faire évaluer chaque contribution par la communauté (≥3 évaluateurs, médiane)	<code>docs/evaluation-explained.md</code> , <code>backend/sql/init.sql:180-209</code>	✓ Vérifié
Produire un corpus exportable prêt pour Whisper	vues <code>whisper_dataset</code> , <code>whisper_training_set</code> , <code>whisper_review_set</code> ; <code>backend/src/routes/dataset-export.ts</code>	✓ Vérifié
Organiser une compétition avec prix en MRU	<code>competitionGateMiddleware</code> , <code>backend/src/services/email.ts:158</code>	✓ Vérifié
Annoter des médias (vidéo/audio découpés en shorts)	<code>backend/src/routes/media.ts</code> , <code>backend/src/services/media-ingest.ts</code>	✓ Vérifié
Relecture experte ("gold standard") + lexique hassaniya	<code>backend/src/routes/reviews.ts</code> , table <code>lexicon</code>	✓ Vérifié
Espace communautaire (sondages, commentaires, feed vocal)	<code>backend/src/routes/community.ts</code>	✓ Vérifié
Veille IA automatisée sur WhatsApp	<code>backend/src/services/ai-news.ts</code> , <code>backend/src/routes/news.ts</code>	✓ Vérifié

1.3 Périmètre technique

✓ **Vérifié** — dépôt **monorepo** à deux applications :

1. Une **SPA/PWA Next.js** (frontend, port 3000)
2. Une **API Express** (backend, port 4000)
3. Une **base PostgreSQL** + un **pooler PgBouncer**
4. Un **reverse proxy Caddy** (TLS automatique)
5. Un **conteneur WAHA** (passerelle WhatsApp)

6. Un **service de sauvegarde** `pg-backup` (profil Docker `backup`)

Le tout orchestré par un unique `docker-compose.yml` , déployé sur **Hetzner Cloud** (`deploy/create-server.sh` : serveur `cx32` , Ubuntu 24.04, Falkenstein).

1.4 Hors périmètre du dépôt

ÉLÉMENT	STATUT
Fichier <code>.env</code> / <code>.env.example</code> (le README en exige la copie)	✗ Absent — cf. §8.4
Tests automatisés (unitaires, intégration, e2e)	✗ Absent — cf. §9.5
Pipeline CI/CD (<code>.github/</code> , <code>.gitlab-ci.yml</code> ...)	✗ Absent
Politique de confidentialité / mentions légales / DPA	✗ Absent — cf. §7.5
Infrastructure as Code (Terraform, Pulumi)	✗ Absent (uniquement des scripts bash <code>hcloud</code>)
Configuration Cloudflare (référéncée dans les commentaires du code)	⚠ À confirmer

2. Stack technique et dépendances principales

2.1 Frontend — package.json

✓ Vérifié

DÉPENDANCE	VERSION	RÔLE
next	16.2.0	Framework (App Router, <code>output: "standalone"</code>)
react / react-dom	19.2.4	UI
typescript	^5	Typage (<code>strict: true</code>)
tailwindcss + @tailwindcss/postcss	^4	CSS
lucide-react	^0.577.0	Icônes
three + @types/three	^0.184	Fond animé (<code>src/components/DottedSurface.tsx</code>)
eslint + eslint-config-next	^9 / 16.2.0	Lint

⚠ À confirmer — @types/three est déclaré en **dependencies** (production) alors qu'il s'agit d'un paquet de types : anomalie mineure de classification.

2.2 Backend — backend/package.json

✓ Vérifié

DÉPENDANCE	VERSION	RÔLE
express	^5.1.0	Serveur HTTP
pg	^8.13.0	Client PostgreSQL (pool)
jsonwebtoken	^9.0.2	JWT HS256
bcryptjs	^2.4.3	Hachage mots de passe (coût 12)
helmet	^8.1.0	En-têtes de sécurité
cors	^2.8.5	CORS strict mono-origine
express-rate-limit	^7.5.0	Limitation de débit (en mémoire)
multer	^2.0.2	Uploads multipart (disque)
zod	^3.24.0	Validation de schémas
nodemailer	^6.9.0	SMTP (Zoho)
cookie-parser	^1.4.7	Cookie <code>rf_token</code> httpOnly
archiver	^7.0.1	Export ZIP du dataset

DÉPENDANCE	VERSION	RÔLE
<code>dotenv</code>	^16.5.0	Chargement <code>.env</code>
<code>tsx</code> (dev)	^4.19.0	Exécution TS (dev + seed)

2.2.1 Anomalies de dépendances constatées

ANOMALIE	PREUVE	IMPACT	STATUT
<code>ipaddr.js</code> importé mais non déclaré	<code>backend/src/middleware/auth.ts:4</code> importe <code>ipaddr.js</code> ; il n'apparaît dans <code>backend/package-lock.json:2433</code> qu'en dépendance transitive de <code>proxy-addr</code> (via Express)	Le build ne fonctionne que par <i>hoisting</i> npm. Si Express/ <code>proxy-addr</code> change d'implémentation, le module disparaît → l'API ne démarre plus , ou la whitelist IP admin casse (échec <i>fail-closed</i> = plus aucun accès admin)	✓ Vérifié
Aucun SDK OpenAI	Appels <code>fetch</code> directs vers <code>https://api.openai.com/v1/...</code> (<code>media.ts:462</code> , <code>reviews.ts:41,50</code> , <code>ai-news.ts:79</code>)	Pas de retry/backoff, pas de gestion de quota structurée (seul un cas 429 est traité dans <code>ai-news.ts:83</code>)	✓ Vérifié
<code>npm install</code> au lieu de <code>npm ci</code>	<code>Dockerfile:5</code> , <code>backend/Dockerfile:5</code>	Le lockfile est ignoré : les builds ne sont pas reproductibles	✓ Vérifié
Images <code>:latest</code> non épinglées	<code>docker-compose.yml:58</code> (<code>edoburu/pgbouncer:latest</code>), <code>:211</code> (<code>devlikeapro/waha:latest</code>)	Risque de chaîne d'approvisionnement : une mise à jour amont peut casser ou compromettre l'environnement	✓ Vérifié
<code>daily-cap.ts</code> = code mort	<code>backend/src/utls/daily-cap.ts</code> : <code>checkDailyCap()</code> n'est appelé nulle part (plafond retiré, cf. <code>validate.ts:66-68</code>)	Confusion / dette	✓ Vérifié
Audit de vulnérabilités des dépendances	Aucun <code>npm audit</code> , Dependabot, Snyk, SBOM	Vulnérabilités connues non surveillées	✗ Absent

2.3 Infrastructure

✓ Vérifié — `docker-compose.yml`

SERVICE	IMAGE	EXPOSITION	NOTES
<code>postgres</code>	<code>postgres:17-alpine</code>	interne uniquement (<code>expose: 5432</code>)	Tuning explicite : <code>shared_buffers=512MB</code> , <code>max_connections=150</code> , limite mémoire 2 Go, <code>shm_size=256mb</code>
<code>pgbouncer</code>	<code>edoburu/pgbouncer:latest</code>	interne	<code>POOL_MODE=transaction</code> , <code>MAX_CLIENT_CONN=500</code> , <code>AUTH_TYPE=trust</code> ⚠
<code>backend</code>	build <code>./backend</code>	interne (<code>4000</code>)	<code>WEB_CONCURRENCY=4</code> , limite mémoire 1 Go, volume <code>recordings:/data/recordings</code>
<code>frontend</code>	build <code>.</code>	interne (<code>3000</code>)	<code>NEXT_PUBLIC_API_URL</code> injecté au build
<code>caddy</code>	<code>caddy:2-alpine</code>	80/443 publiés	Monte <code>recordings</code> en lecture seule

SERVICE	IMAGE	EXPOSITION	NOTES
pg-backup	postgres:17-alpine	interne, profil backup	Dump DB + tar audio, rétention 7/3
waha	devlikeapro/waha:latest	interne (3000)	Moteur NOWEB , volume waha_sessions

Binares système requis dans l'image backend (☒ backend/Dockerfile:16-18) : ffmpeg , ffprobe , python3 , yt-dlp (téléchargé depuis GitHub latest au build — non épinglé).

3. Structure du dépôt et rôle des dossiers

✓ **Vérifié** — arborescence réelle (hors `node_modules/`, `.next/`)


```

elson-main/
├── src/                                     # — FRONTEND (Next.js App Router) —
│   ├── app/
│   │   ├── page.tsx                       #   Accueil applicatif (505 l.)
│   │   ├── layout.tsx                     #   Root layout, thème pré-paint, i18n, PWA
│   │   ├── login/                         #   Connexion + inscription + OTP WhatsApp
│   │   ├── onboarding/                    #   Guide obligatoire avant participation
│   │   ├── contribute/                     #   Traduction + enregistrement audio (626 l.)
│   │   ├── validate/                       #   Évaluation + vote comparatif (514 l.)
│   │   ├── leaderboard/                   #   Classement (codes anonymes rotatifs)
│   │   ├── community/                     #   Feed communautaire immersif
│   │   ├── dashboard/, credits/           #   Profil, langue ; crédits contributeurs
│   │   ├── my-agenda/                     #   Programme (créneaux / crédit temps)
│   │   ├── faq/, rules/                   #   Contenu statique i18n
│   │   ├── verify-email/, reset-password/
│   │   ├── studio/                         #   Vitrine "modèle en création"
│   │   ├── admin/                         #   Console admin (1556 l. + 11 panneaux)
│   │   ├── reviewer/                      #   Relecture "gold" (admin)
│   │   ├── media-studio/                  #   Ingestion vidéo/audio (admin)
│   │   ├── eval-control/                  #   Pilotage de l'évaluation (admin)
│   │   └── news-bot/                      #   Bot de veille IA WhatsApp (admin)
│   ├── components/                        # 23 composants (AudioRecorder, HassaniyaInput,
│   │   └──                                     #   ScheduleBoard, CreditMeter, Navigation...)
│   └── lib/
│       ├── api.ts                         #   Client API unique (1463 l.) – tokens en mémoire
│       ├── i18n.ts / i18n-context.tsx     #   Traductions FR/EN/AR (2067 l.)
│       ├── fingerprint.ts                 #   Empreinte de device (anti multi-comptes)
│       ├── outbox-db.ts                   #   File d'attente durable IndexedDB (audio)
│       └── gamification.ts, hassaniya.ts
├── backend/                               # — BACKEND (Express / TypeScript ESM) —
│   ├── src/
│   │   ├── server.ts                     #   Point d'entrée : cluster, middlewares globaux,
│   │   │   └──                                     #   5 workers de fond, /recordings, /api/health
│   │   ├── config.ts                     #   Chargement/validation des variables d'env
│   │   ├── db.ts                         #   Pool pg + helpers query/queryOne/execute/transaction
│   │   ├── middleware/
│   │   │   ├── auth.ts                   #   JWT, rôle, session unique, admin+IP, gates horaires
│   │   │   ├── security.ts               #   Rate limiters, anti-scraping, anti-vitesse
│   │   │   └── validate.ts               #   Middleware Zod
│   │   ├── routes/                       #   11 routeurs, 225 endpoints
│   │   ├── services/                     #   audio-processor, media-ingest, email, whatsapp,
│   │   │   └──                                     #   alert, geoip, ai-news, phrase-pipeline
│   │   ├── utils/                        #   api-key, audio-token, dataset-token, anon, otp,
│   │   │   └──                                     #   fraud, audit, cache, schedule, credit-active...
│   │   └── scripts/                       #   seed-flores.ts, test-v5.ts (scripts manuels)
│   ├── sql/
│   │   ├── init.sql                      #   Schéma initial (8 tables seulement)
│   │   ├── seed.sql                      #   20 groupes trilingues de démonstration
│   │   └── migration_v2..v73_*.sql       #   72 migrations additives
│   └── Dockerfile, docker-entrypoint.sh

```

```

├─ deploy/                                # Scripts d'exploitation (bash / mjs)
│   ├── create-server.sh                 #   Provisionnement Hetzner via `hcloud`
│   ├── setup-server.sh                 #   Durcissement OS (SSH 2222, UFW, fail2ban...)
│   ├── deploy.sh                       #   rsync + docker compose build/up
│   ├── disk-alert.sh                   #   Alerte disque par mail (cron horaire)
│   └─ daily-leaderboard*.mjs, top10_email.html
├─ docs/                                # evaluation-explained.md, quality-scoring.md
├─ public/                              # _landing.html (130 Ko, servi à `/'`), sw.js,
                                        #   manifest.json, logos SVG, 8 samples audio
├─ scripts/seed_flores.py                # Import FLORES (Python)
├─ docker-compose.yml, Caddyfile, Dockerfile
├─ next.config.ts, tsconfig.json, eslint.config.mjs, postcss.config.mjs
└─ architecture.excalidraw              # Schéma d'architecture (source Excalidraw)

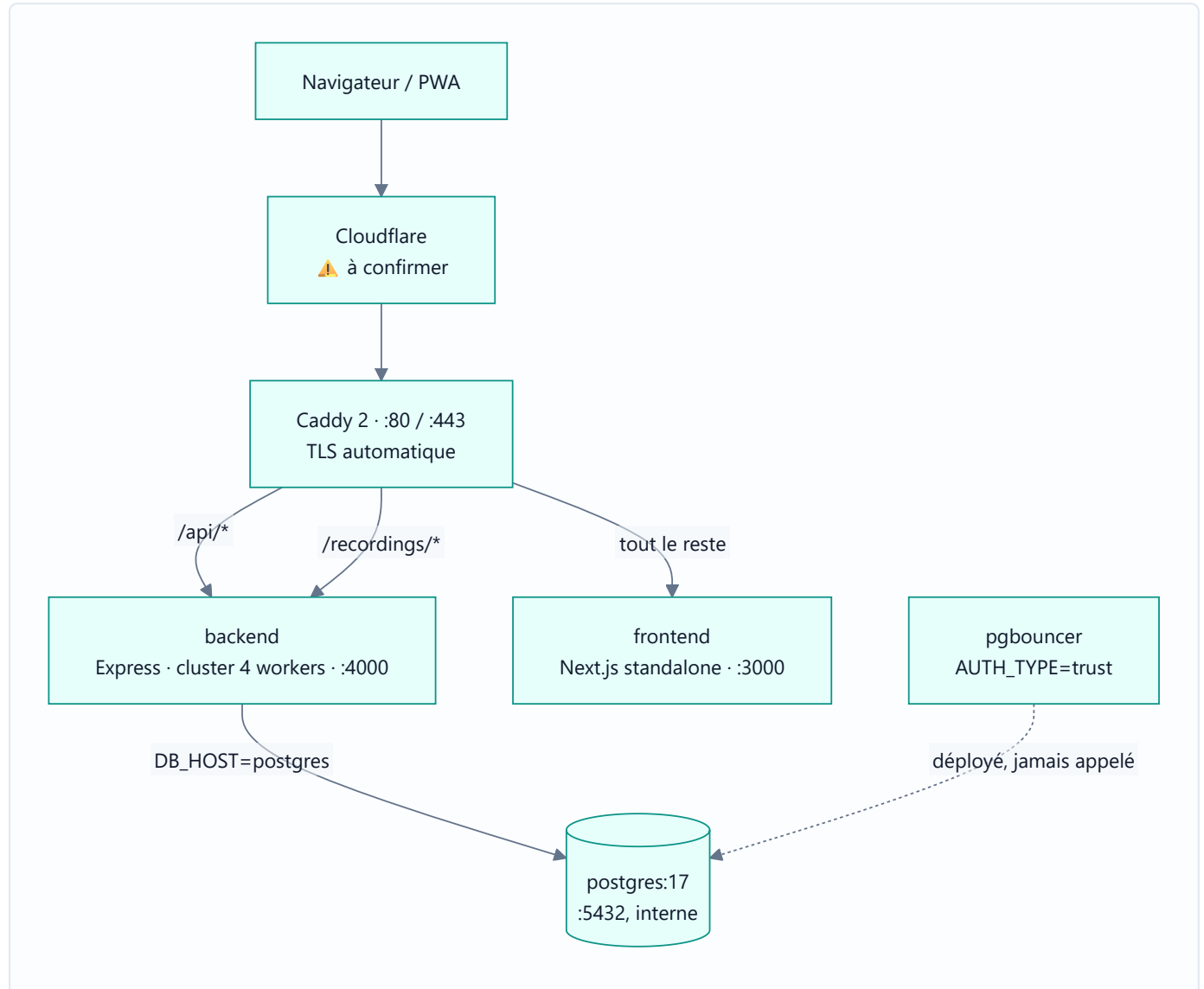
```

Point notable ✅ `next.config.ts:9-13` : la route `/` est **réécrite** vers `/_landing.html` (page marketing statique de 130 Ko), l'application démarre donc réellement sur `/contribute` ou `/login`.

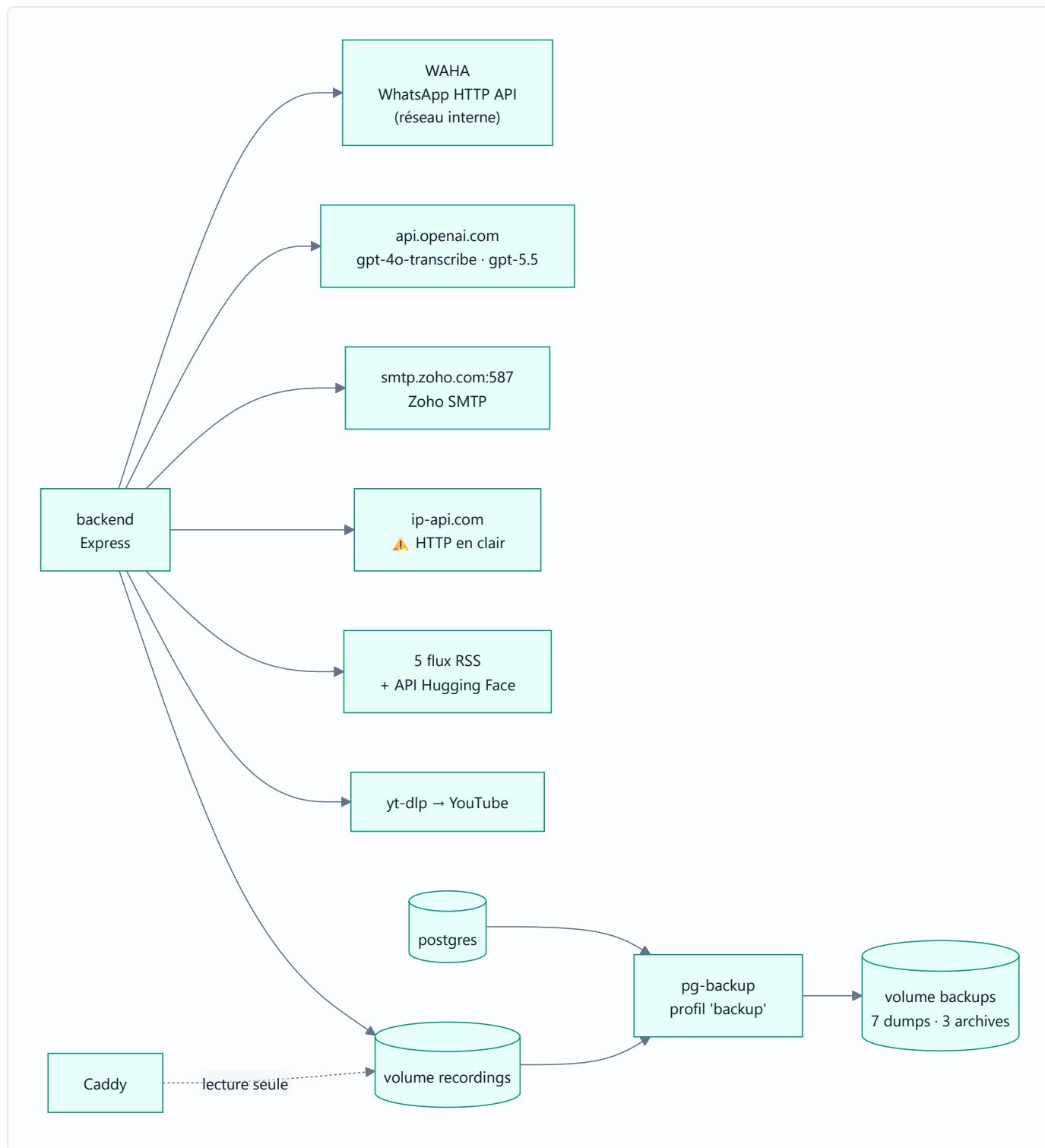
4. Architecture

4.1 Vue de déploiement

(a) Chemin des requêtes entrantes



(b) Sorties réseau, stockage et sauvegardes



✓ **Vérifié** — `docker-compose.yml`, `Caddyfile`

⚠ **Constat important** : `docker-compose.yml:101-102` fixe `DB_HOST: postgres` / `DB_PORT: "5432"` pour le backend. Le backend **contourne donc PgBouncer** et se connecte directement à PostgreSQL avec un pool de 100 connexions par worker (`DB_MAX_CONNECTIONS: "100"` × 4 workers = 400 connexions théoriques face à `max_connections=150`). PgBouncer tourne inutilement, avec `AUTH_TYPE=trust` .

4.2 Architecture frontend

✓ **Vérifié**

- **Next.js App Router**, `output: "standalone"` (`next.config.ts:4`), `reactStrictMode: false` (désactivé volontairement pour ne pas doubler les timers de polling — `next.config.ts:5-8`).
- **Tout composant de page est "use client"** : l'application est en pratique une SPA ; aucun *Server Component* de données, aucune *Route Handler* Next.js, aucun *Server Action*. Toute la donnée transite par `src/lib/api.ts` .
- **Client API unique** (`src/lib/api.ts`) : ajoute systématiquement `X-Requested-By: elson-web` , `X-API-Key` (clé de session en mémoire) et `Authorization: Bearer <accessToken>` (mémoire), avec `credentials: "include"` .
Rafraîchissement *single-flight* sur 401 `TOKEN_EXPIRED` (`api.ts:63-72`).
- **PWA** : `public/manifest.json` + `public/sw.js` (cache `hassaniya-v49-...` , stratégie *network-first* pour la navigation, *cache-first* pour les assets ; `/api/*` et `/recordings/*` **jamais** mis en cache — `sw.js:46-53`).
- **Résilience offline** : `src/lib/outbox-db.ts` persiste chaque contribution (texte + blob audio) dans **IndexedDB** *avant* tout appel réseau, et rejoue à la reprise.
- **i18n** : FR / EN / AR avec support RTL, 2067 lignes de traductions (`src/lib/i18n.ts`).
- **Thème** clair/sombre appliqué avant le premier paint via un script inline (`layout.tsx:49-53`).

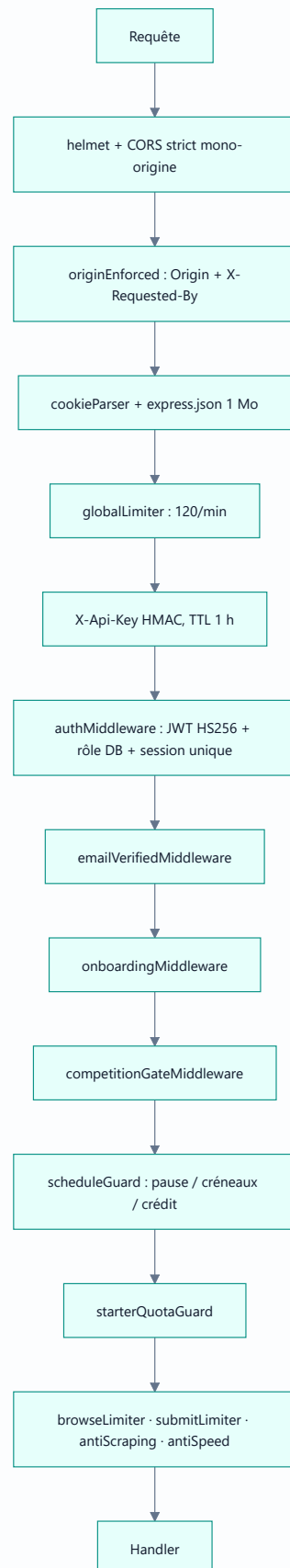
4.3 Architecture backend

✓ **Vérifié** — `backend/src/server.ts`

- **Mode cluster** : en production, le process primaire fork `WEB_CONCURRENCY` workers (défaut : `min(cœurs, 4)`) et les redémarre automatiquement en cas de sortie (`server.ts:29-49`).
- **Invalidation de cache inter-workers** : chaque worker a son cache mémoire (`utils/cache.ts`) ;
`invalidateGlobal()` diffuse via IPC (`process.send` → primaire → tous les workers) — utilisé pour `session_id` après login.
- **5 workers de fond** (démarrés dans chaque worker HTTP, avec verrouillage atomique via `competition_config` pour les tâches à exécution unique) :

WORKER	FRÉQUENCE	RÔLE	RÉF.
<code>startNotificationDispatcher</code>	15 s, lots de 20	Envoi email + WhatsApp (<code>FOR UPDATE SKIP LOCKED</code>)	<code>server.ts:401</code>
<code>startAutoActivation</code>	60 s	Bascule <code>competition_status</code> et <code>is_activated</code> par date	<code>server.ts:549</code>
<code>startCollusionDetector</code>	1 h	Détection de paires validateur→contributeur suspectes (+30 <code>fraud_score</code> , alerte mail)	<code>server.ts:614</code>
<code>startNewsBot</code>	60 s / 90 min	Digest IA + alertes "hot" sur WhatsApp	<code>server.ts:689</code>
<code>startTokenPurge</code>	6 h	Purge des <code>refresh_tokens</code> révoqués/expirés	<code>server.ts:724</code>
<code>resumeOrphanedIngests</code>	au boot	Reprise des jobs d'ingestion média interrompus	<code>services/media-ingest.ts:209</code>

- **Gestion d'erreur centralisée** (`server.ts:375-390`) : l'erreur technique va aux logs, le client reçoit un message générique FR + `code: "TEMP_ERROR"` (aucune fuite de stack).
- **Chaîne de gardes** appliquée aux routes de participation :



✓ **Vérifié** — `routes/phrases.ts:20-91` , `routes/validate.ts:24-28` , `server.ts:113-164`

4.4 Architecture base de données

✓ Vérifié

- PostgreSQL 17, extensions `uuid-oss` et `pgcrypto` (`init.sql:7-8`).
- Accès exclusivement paramétré via `backend/src/db.ts` (`pool.query(text, params)`).
- Logique métier partiellement en base — triggers et fonctions : `create_profile_on_signup`, `update_contribution_after_validation` (auto-approbation à ≥ 3 validations et moyenne $\geq 3,5$; rejet si $< 2,5$), `update_streak`, `update_badges`, `update_user_level`, `update_quality_after_validation`, `wmedian` (médiane pondérée), `validator_weight`, `point_mult`, `dq_recompute_contribution`, `protect_identity_fields`, `expire_stale_locks`, `get_best_translation`.
- `protect_identity_fields` (✓ `migration_v4_identity.sql`) : trigger `BEFORE UPDATE` sur `users` qui lève une exception si `nni`, `whatsapp`, `first_name`, `last_name` ou `birthdate` sont modifiés après l'inscription — excellente mesure anti-usurpation.
- 17 vues dont `leaderboard`, `quality_leaderboard`, `admin_user_list`, `public_stats`, `signup_forensics`, `collusion_detector`, `reciprocity_pairs`, `bot_signals`, `competition_fairness`, `whisper_*`.
- Table de configuration dynamique `competition_config` (clé/valeur) : ~80 clés pilotant le comportement à chaud (statut compétition, pauses, stratégie de planning, multiplicateurs de points, bascule qualité, bot news, disclaimer, etc.).

4.4.1 Gestion des migrations — risque majeur

CONSTAT	PREUVE	STATUT
<code>init.sql</code> ne crée que 8 tables (<code>users</code> , <code>profiles</code> , <code>phrases</code> , <code>contributions</code> , <code>validations</code> , <code>tags</code> , <code>contribution_tags</code> , <code>phrase_locks</code> , <code>refresh_tokens</code>) sur les ~55 utilisées	<code>backend/sql/init.sql</code>	✓ Vérifié
72 migrations doivent être appliquées manuellement, une par une	<code>README.md:136-140</code> (<code>docker compose exec -T postgres psql ... -f -</code>)	✓ Vérifié
Aucun outil de migration (pas de table de version, pas de runner, pas d'ordre garanti)	—	✗ Absent
10 numéros de version en doublon : v25, v37, v38, v39, v40, v48, v49, v50, v53, v54 (chacun x2 fichiers différents)	<code>ls backend/sql/</code>	✓ Vérifié
Le code anticipe la dérive de schéma : plusieurs handlers rattrapent les codes PostgreSQL <code>42P01</code> (table absente) et <code>42703</code> (colonne absente)	<code>community.ts:35,51,1060</code> , <code>validate.ts:408</code> , <code>phrases.ts</code> (<code>try/catch</code> sur <code>lexicon</code> , <code>media_transcriptions</code>)	✓ Vérifié
● Trois objets de base de données utilisés par le code n'ont AUCUNE définition dans le dépôt (voir §4.4.2)	recherche exhaustive dans <code>backend/sql/*.sql</code>	✓ Vérifié

➔ Conséquence : un environnement neuf (recette, GCP, poste de dev) n'est pas reproductible de façon fiable. C'est le principal frein technique à un redéploiement.

4.4.2 ● Objets de base de données manquants du dépôt

✓ Vérifié — recherche exhaustive de `CREATE TABLE` / `CREATE VIEW` sur les 74 fichiers de `backend/sql/` :

OBJET	UTILISÉ PAR LE CODE	DÉFINITION DANS LE DÉPÔT
<code>votes</code> (table)	<code>INSERT INTO votes</code> (<code>validate.ts:510</code>), <code>SELECT ... FROM votes</code> (<code>validate.ts:230,501</code>), vue <code>leaderboard</code> via <code>profiles.total_votes</code>	● AUCUNE
<code>votable_phrases</code> (vue)	<code>SELECT vp.* FROM votable_phrases vp</code> (<code>validate.ts:227</code>) — cœur de la phase de vote comparatif	● AUCUNE
<code>season0_contributors</code>	<code>SELECT username, contributions FROM season0_contributors</code> sur l'endpoint <code>public /api/public/stats</code> (<code>server.ts:300</code>)	● AUCUNE — seulement mentionnée dans un commentaire (<code>migration_v23_season0_reset.sql:19</code> : « already lives in <code>season0_contributors</code> »)

Implication directe : ces objets ont été créés **manuellement en production**, hors versionnement. Sur un environnement reconstruit à partir du dépôt :

- toute la **fonctionnalité de vote comparatif** échoue (`42P01`) — et contrairement à d'autres endroits, `validate.ts:226-235` **ne rattrape pas** cette erreur, elle remonte donc en 500 ;
- `/api/public/stats` est protégé par un `.catch(() => [])` (`server.ts:302`) et dégrade silencieusement.

💡 **Correctif indispensable avant toute migration** : extraire le schéma réel de la production (`pg_dump --schema-only`) et en faire le schéma de référence versionné.

4.5 Services externes

SERVICE	USAGE	PROTOCOLE	RÉF.	STATUT
OpenAI	Transcription (<code>gpt-4o-transcribe</code>), traduction & curation (<code>gpt-5.5</code>)	HTTPS, clé <code>OPENAI_API_KEY</code>	<code>media.ts:462</code> , <code>reviews.ts:41,50</code> , <code>ai-news.ts:79</code>	✓ Vérifié
WAHA (WhatsApp)	OTP d'inscription/reset, notifications, digest, médias	HTTP interne + <code>X-Api-Key</code>	<code>services/whatsapp.ts</code>	✓ Vérifié
SMTP Zoho	Vérification email, reset, bienvenue, notifications, alertes sécurité	SMTP 587 STARTTLS	<code>services/email.ts</code> , <code>services/alert.ts</code>	✓ Vérifié
ip-api.com	Géolocalisation IP à l'inscription (proxy/hosting/pays → score de fraude)	⚠ HTTP en clair	<code>services/geoip.ts:40-43</code>	✓ Vérifié
Google Fonts	Inter, Cairo, Noto Kufi Arabic	HTTPS	<code>layout.tsx:54-59</code> , <code>public/_landing.html</code>	✓ Vérifié
yt-dlp / YouTube	Ingestion de vidéos pour l'annotation	HTTPS	<code>services/media-ingest.ts:123</code>	✓ Vérifié
Flux RSS	5 sources IA pour le bot de veille	HTTPS	<code>services/ai-news.ts:10-16</code>	✓ Vérifié
Hugging Face API	Modèles tendance pour le digest	HTTPS	<code>services/ai-news.ts:62</code>	✓ Vérifié

5. Fonctionnalités, rôles utilisateurs et parcours

5.1 Rôles

✓ Vérifié

RÔLE / ATTRIBUT	SOURCE	EFFET
<code>user</code>	<code>users.role</code> (défaut)	Contributeur standard
<code>admin</code>	<code>users.role</code>	Accès à ~165 endpoints admin (rôle relu en base à chaque requête, jamais depuis le JWT) + whitelist IP obligatoire
<code>moderator</code>	Autorisé par la contrainte <code>CHECK</code> de <code>init.sql:20</code> mais aucun code ne l'utilise	Rôle mort
<code>eval_exempt</code>	<code>profiles.eval_exempt</code>	"Relecteur" : évalue librement, ignore le seuil de contributions, exclu du classement
<code>leaderboard_hidden</code>	<code>profiles.leaderboard_hidden</code>	Masqué du classement et des rangs
<code>disqualified</code>	<code>profiles.disqualified</code>	Disqualifié (points/qualité/confiance sauvegardés avant DQ)
<code>community_blocked</code>	<code>profiles.community_blocked</code>	Interdit de publier/commenter
<code>credit_blocked</code>	<code>profiles.credit_blocked</code>	Crédit temps suspendu
<code>is_activated</code>	<code>profiles.is_activated</code>	Autorisé à participer (modes <code>open</code> / <code>invite_only</code> / <code>date_auto</code>)
<code>account_tag</code>	<code>profiles.account_tag</code>	Étiquetage administratif

5.2 Fonctionnalités par domaine

Contribution (`/contribute`)

✓ Vérifié — `routes/phrases.ts`, `services/phrase-pipeline.ts`

- **Pipeline de distribution intelligent** : priorité aux phrases assignées par un admin, puis score composite = couverture (0/1/2/3+ traductions) + phase du corpus (`coverage` / `quality` / `polish`) + diversité de catégorie + aléa, tirage pondéré dans le top 5, verrouillage DB 5 min (`phrase_locks`).
- **Anti-triche bidirectionnel** : une phrase déjà traduite (ou verrouillée) par l'utilisateur n'est jamais proposée à l'évaluation, et réciproquement.
- **Soumission atomique** texte + audio (audio **obligatoire**) : validation magic-bytes, déduplication SHA-256 de l'audio par utilisateur, contrôles durée (1,5 s – 120 s) et taille (≥5 Ko, ≤10 Mo), post-traitement FFmpeg (WAV 16 kHz mono, high-pass 80 Hz, loudnorm –16 LUFS, noise gate, rejet du silence à –35 dB).
- **Idempotence** : un rejet de la file d'attente renvoie `success` sans re-créditer.
- **Coffre "aucun travail perdu"** : toute soumission rejetée est archivée (`failed_submissions`) + fichier déplacé sous `<uploadDir>/failed/`) pour re-création manuelle par un admin.

- **Assistance à la saisie** : `HassaniyaInput` souligne les mots hors lexique, propose des corrections (distance de Levenshtein ≤ 2) et de l'autocomplétion (lexique communautaire de 12 000 mots, cache 1 h).
- **Barème** : 15 points par contribution (+5 possible via trigger de série).

Évaluation & vote (`/validate`)

✓ **Vérifié** — `routes/validate.ts`

- Deux modes servis par le même endpoint : **évaluation** (note texte 1–5 + clarté audio 1–5 + validité) puis **vote comparatif** entre traductions approuvées.
- Contributeur **anonymisé** pour les évaluateurs (code opaque HMAC visible des admins seulement).
- Audio **masqué** aux comptes de niveau <2 (sauf relecteurs) ; sinon **URL signée HMAC 10 min liée à l'utilisateur**.
- Un "Reject" force les notes au minimum côté serveur (`effTextAccuracy = 1`).
- Barème : 3 points/évaluation, 2 points/vote, × multiplicateur de langue actif.
- **Correction "gold"** possible (`POST /correct`) : texte corrigé + ré-enregistrement audio.

Relecture experte (`/reviewer`) — admin

✓ **Vérifié** — `routes/reviews.ts` : file de relecture par contributeur, texte corrigé, correction de la phrase source, alimentation du **lexique validé**, mode vidéo par segments avec pré-annotation IA (transcription + traduction).

Media Studio (`/media-studio`) — admin

✓ **Vérifié** — `routes/media.ts` , `services/media-ingest.ts` : ingestion YouTube (yt-dlp) ou upload (jusqu'à 800 Mo, ou par *chunks* de 30 Mo pour contourner la limite Cloudflare), **découpage intelligent sur les silences** (FFmpeg `silencedetect`), max 300 segments, nettoyage automatique des clips sans parole, **pièges "honeypot"** (faux brouillon IA pour détecter les correcteurs qui ne corrigent pas), reconstitution du transcript complet.

Communauté (`/community`)

✓ **Vérifié** — `routes/community.ts` : cartes officielles/propositions (1/2 h), sondages, commentaires, `@mentions` avec notification, likes, **feed vocal immersif anonymisé**, "exemple à imiter" publié par l'admin. Publication conditionnée par un gate `top N` du classement, un mode `admin_only` , et un blocage individuel.

Compétition, planning et crédit temps

✓ **Vérifié** — `middleware/auth.ts` , `utils/schedule.ts` , `utils/credit-active.ts`

Trois stratégies d'ouverture, pilotées à chaud :

1. `slots` — créneaux par date/heure/langue (`schedule_slots`), avec bascule automatique si la file d'évaluation est vide.
2. `credit` — budget horaire quotidien par activité, fenêtre démarrée au premier usage.
3. `hybrid` — créneaux gratuits + crédit personnel *facturé à la seconde* hors créneau, avec pause/reprise manuelle et facturation côté serveur sur **chaque action réelle** (bloquer les pings client ne stoppe pas le compteur).

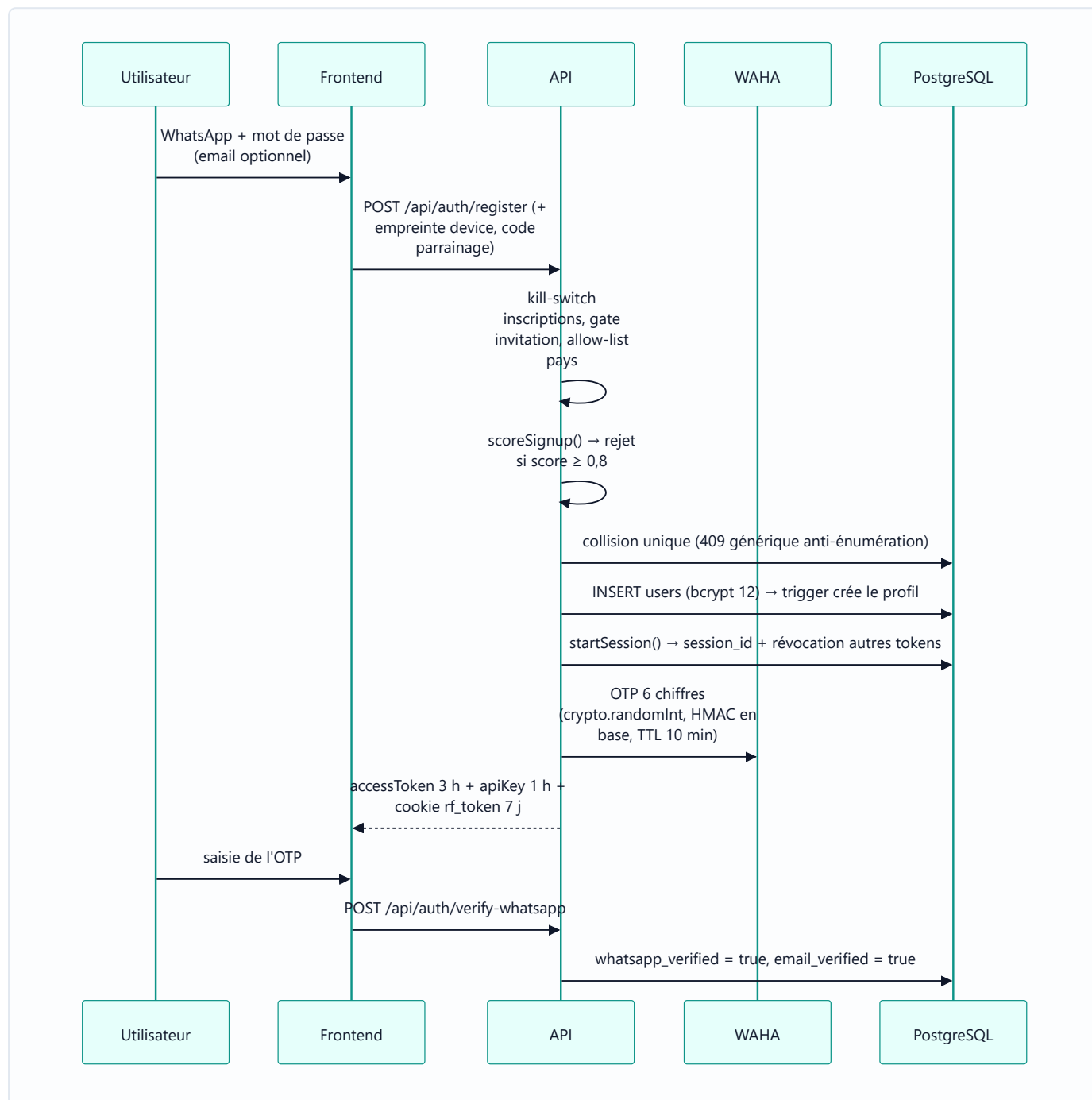
Plus : **pause qualité** (kill-switch ponctuel), **tournois flash** (isolation mode B : 0 point officiel, classement séparé, volet "ambassadeur" via parrainage).

Autres

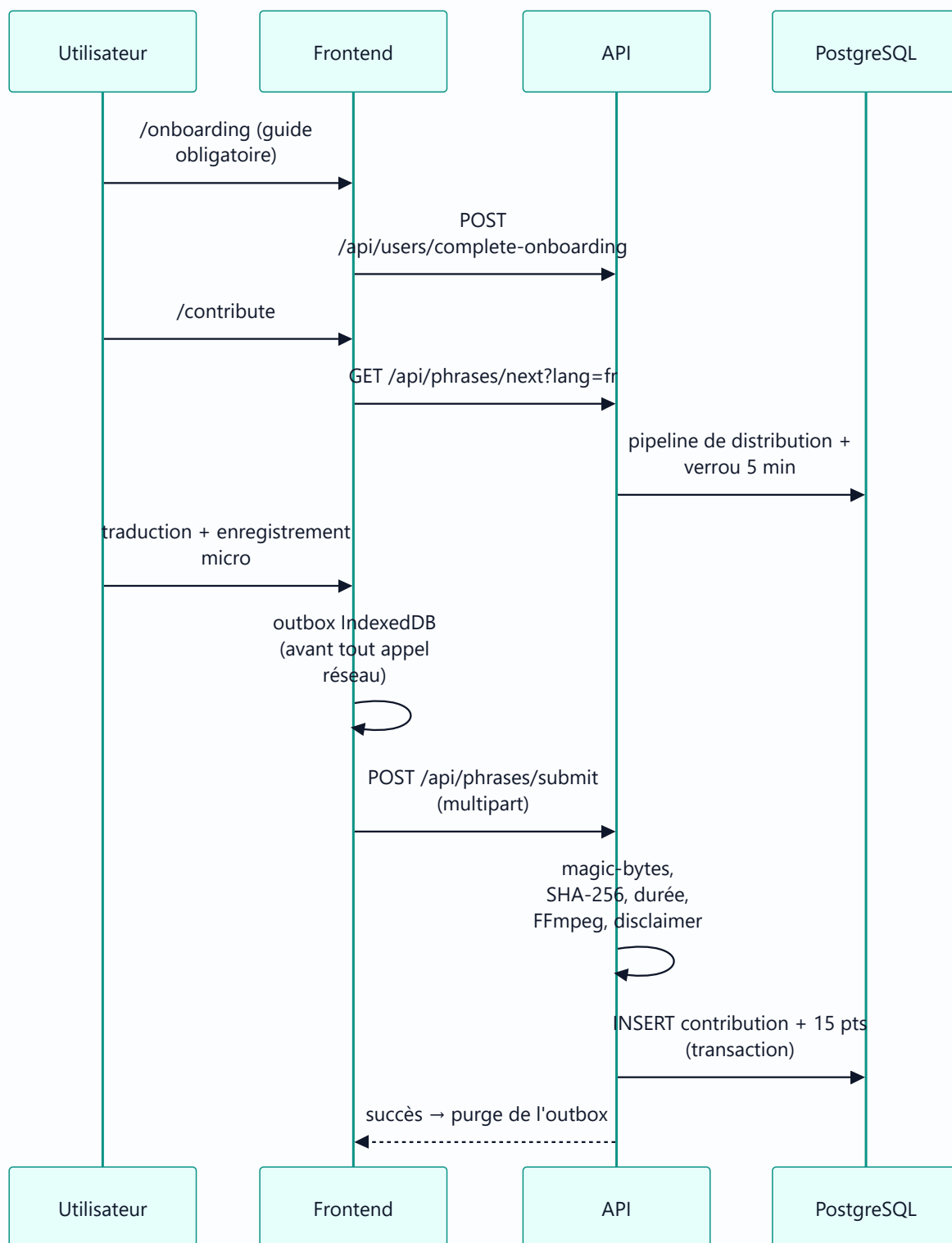
✅ **Vérfié** — parrainage à commission dégressive ([utils/referral.ts](#)), crédits contributeurs (citer ses aidants), notifications in-app + email + WhatsApp avec médias, listes de diffusion, export dataset ZIP/CSV, disclaimer versionné, bot de veille IA.

5.3 Parcours principaux

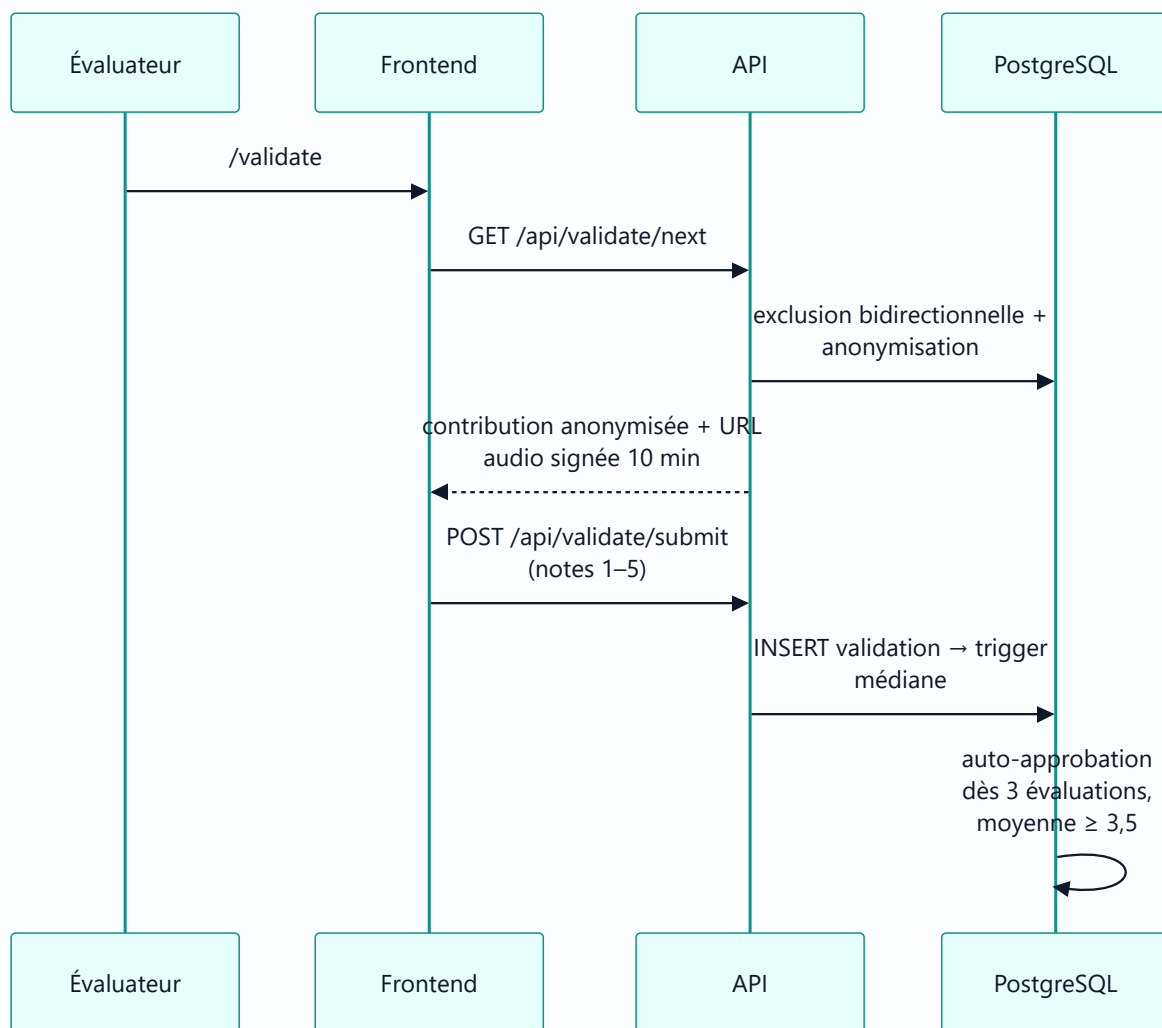
(1) Inscription et vérification du numéro



(2) Onboarding puis contribution (texte + audio)



(3) Évaluation croisée



6. APIs, authentification, autorisation et gestion des sessions

6.1 Inventaire des endpoints

✓ **Vérifié** — 225 routes HTTP réparties en 11 routeurs :

ROUTEUR	PRÉFIXE	ENDPOINTS	GARDE AU NIVEAU DU ROUTEUR
auth.ts	/api/auth	14	authLimiter global ; auth par route
phrases.ts	/api/phrases	9	auth + email + onboarding + gate compétition + scheduleGuard("contribute") + quota starter
validate.ts	/api/validate	5	auth + email + onboarding + gate compétition + scheduleGuard("validation")
users.ts	/api/users	106	authMiddleware ; adminMiddleware par route
community.ts	/api/community	42	authMiddleware (sauf /flash/active , public)
media.ts	/api/media	29	authMiddleware ; adminMiddleware sauf /next , /submit , /lexicon , /mode
reviews.ts	/api/reviews	8	auth + admin (routeur entier)
news.ts	/api/news	6	auth + admin (routeur entier)
eval-admin.ts	/api/eval	3	auth + admin (routeur entier)
credits.ts	/api/credits	3	authMiddleware ; /admin/all en admin
dataset-export.ts	/api/dataset-zip	1 handler	token signé uniquement (hors auth)

~165 endpoints sont protégés par adminMiddleware (rôle + whitelist IP).

Endpoints publics (sans authentification)

✓ **Vérifié** — server.ts:136-164

ENDPOINT	CONTENU EXPOSÉ
GET /api/health	État DB + horodatage
GET /api/public/stats	Statistiques globales, config compétition, jusqu'à 300 pseudos d'utilisateurs en ligne, liste des contributeurs "saison 0" (pseudo + nombre de contributions)
GET /api/community/flash/active	Tournoi flash actif (titre, dates, prix)
POST /api/auth/*	Inscription, login, refresh, reset (14 routes)
GET /api/dataset-zip	Corpus validé complet + audio, protégé par token signé (cf. §11)

Obfuscation des chemins admin

✓ **Vérfié** — `server.ts:350-360` : les appels admin passent par `/api/m/*` qui est réécrit en interne vers `/admin/*`, « pour éviter que "/admin/" apparaisse dans le bundle JS ». C'est de la **sécurité par obscurité** — l'autorisation réelle repose bien sur `adminMiddleware`.

6.2 Authentification

✓ **Vérfié**

MÉCANISME	DÉTAIL	FICHIER
Identifiants de connexion	WhatsApp ou NNI (10 chiffres) ou email + mot de passe	<code>auth.ts:398-464</code>
Mot de passe	bcrypt, coût 12 ; minimum 6 caractères , aucune règle de complexité	<code>auth.ts:126,280,899</code>
Comparaison à temps constant simulé	<code>bcrypt.hash()</code> factice si l'utilisateur n'existe pas	<code>auth.ts:427</code>
Access token	JWT HS256 , <code>exp</code> 3 h (10800 s codé en dur), claims <code>sub</code> , <code>email</code> , <code>role</code> , <code>sid</code>	<code>middleware/auth.ts:504-510</code>
Refresh token	JWT HS256 <code>type:"refresh"</code> , <code>jti</code> UUID, <code>7j</code> , cookie <code>rf_token</code> : <code>httpOnly</code> , <code>sameSite=strict</code> , <code>secure</code> en prod, <code>path=/api/auth</code>	<code>auth.ts:20-47</code>
Stockage refresh	SHA-256 en base (<code>refresh_tokens.token_hash</code>), purge toutes les 6 h	<code>auth.ts:39-44</code> , <code>server.ts:724</code>
Rotation + détection de réutilisation	Un token révoqué déclenche la révocation globale (RFC 6749) sauf si même session ou dans une fenêtre de grâce de 3 h	<code>auth.ts:529-562</code>
Session unique par compte	<code>profiles.session_id</code> (UUID) comparé au claim <code>sid</code> ; un login révoque tous les refresh tokens et invalide le cache profil sur tous les workers	<code>auth.ts:59-68</code> , <code>middleware/auth.ts:116-119</code>
Clé de session <code>X-Api-Key</code>	HMAC-SHA256 (<code>userId:timestamp</code>), TTL 1 h , comparaison <code>timingSafeEqual</code> . Exigée sur tout <code>/api/*</code> en production	<code>utils/api-key.ts</code>
OTP WhatsApp	6 chiffres via <code>crypto.randomInt</code> , HMAC-SHA256 avec poivre serveur , TTL 10 min, cooldown 60 s, max 5/h, max 5 tentatives, comparaison à temps constant	<code>utils/otp.ts</code>
Vérification email / reset par email	code 6 chiffres <code>Math.random()</code> , SHA-256 sans poivre , TTL 30/15 min, max 5 tentatives	<code>auth.ts:729-733</code> , <code>auth.ts:805-811</code>
Algorithme JWT explicite	<code>{ algorithms: ["HS256"] }</code> partout — protège contre la confusion d'algorithme	<code>middleware/auth.ts:77</code> , <code>auth.ts:484</code> , <code>server.ts:200</code> , <code>security.ts:23</code>
Rôle jamais lu depuis le JWT	Toujours relu en base (cache 10 s par utilisateur)	<code>middleware/auth.ts:85-105</code>
2FA / MFA pour les admins	—	✗ Absent

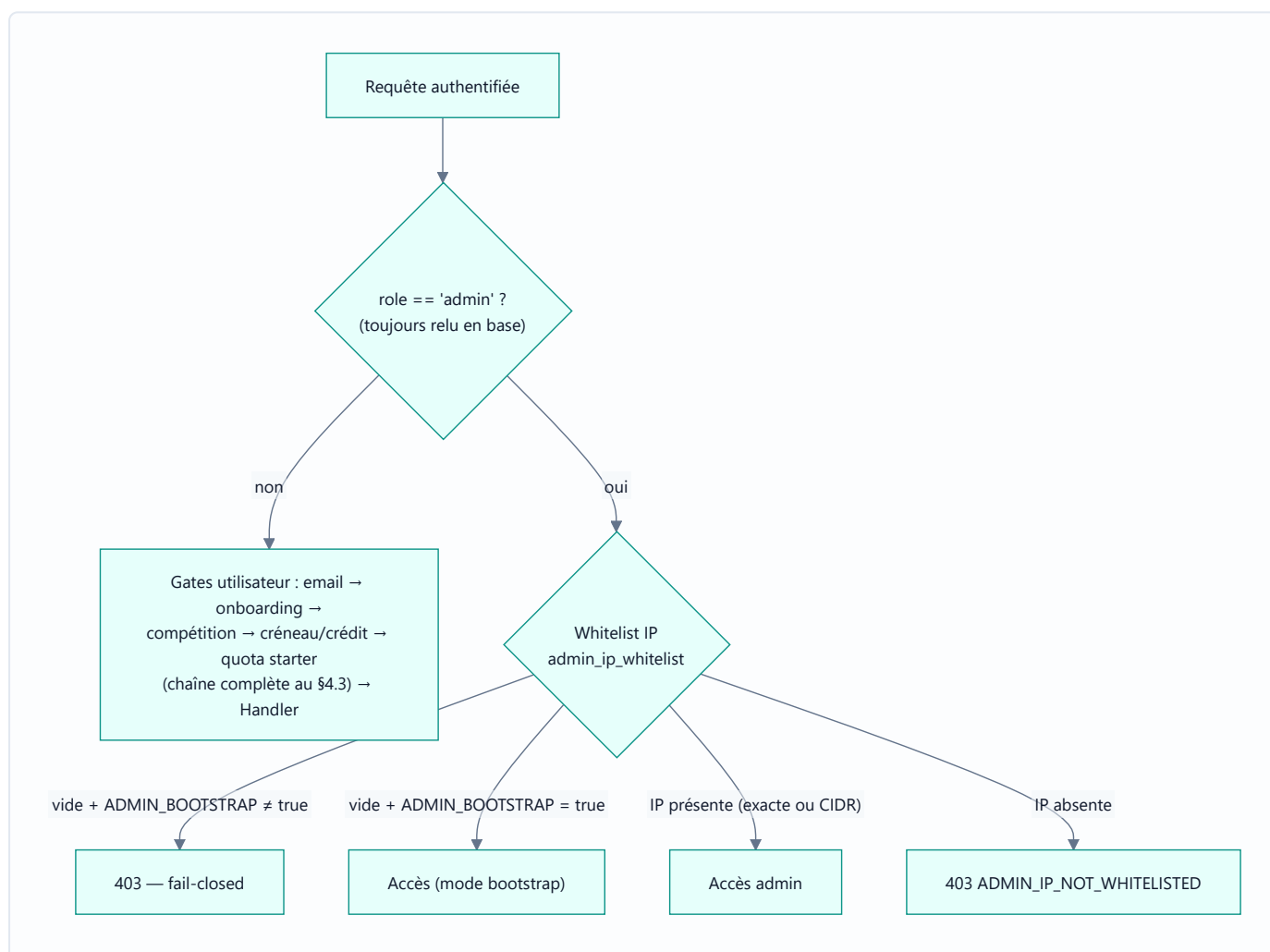
MÉCANISME	DÉTAIL	FICHIER
Verrouillage de compte après N échecs	—	✗ Absent (seul un plafond par IP existe)
CAPTCHA à l'inscription	—	✗ Absent

Incohérence relevée

⚠ `config.jwtAccessExpiresIn` et `config.jwtRefreshExpiresIn` (`config.ts:30-31` , variables `JWT_ACCESS_EXPIRES` / `JWT_REFRESH_EXPIRES`) sont **définis mais jamais utilisés** : `generateAccessToken` / `generateRefreshToken` codent en dur `10800` et `604800` . Modifier ces variables d'environnement **n'a aucun effet**.

6.3 Autorisation

✓ Vérifié



Points forts :

- La whitelist IP admin est **fail-closed** par défaut (`middleware/auth.ts:211-219`).
- Support **CIDR IPv6** (essentiel pour les préfixes mobiles rotatifs).

- L'IP réelle est lue dans `CF-Connecting-IP` (non usurpable derrière Cloudflare), avec repli sur `req.ip` (`middleware/auth.ts:224-229`).
- Le mode `invisible_validator`, la portée d'évaluation (`eval_scope_users`) et le mode "relecteurs uniquement" offrent un contrôle fin.

Point d'attention : ⚠ Le repli `req.ip` s'applique si `CF-Connecting-IP` est absent. Avec `app.set("trust proxy", 1)` (`server.ts:56`), `req.ip` vient du dernier proxy de confiance. Si l'application était exposée **sans** Cloudflare/Caddy devant, un client pourrait influencer `X-Forwarded-For`. ⚠ **À confirmer** au niveau infrastructure.

6.4 Protection CSRF

✅ **Vérifié** — défense en profondeur à 4 niveaux :

1. Cookie `sameSite: "strict"` + `path: "/api/auth"` — il n'est envoyé que sur les routes d'authentification, jamais en contexte cross-site.
2. `originEnforced` : toute méthode ≠ GET exige un `Origin` allow-listé **et** `X-Requested-By: elson-web` (en-tête personnalisé qu'aucun site tiers ne peut poser sans préflight CORS, lui-même bloqué) — `server.ts:87-111`.
3. CORS mono-origine (`allowedOrigins = [config.frontendUrl]`).
4. Access token en **mémoire JS uniquement** (jamais en cookie) : il n'est pas envoyé automatiquement par le navigateur.

➔ **Risque CSRF : faible.** ⚠ Réserve : `originEnforced` et le gate `X-API-Key` sont **entièrement désactivés hors production** (`server.ts:89`, `server.ts:136`). La valeur par défaut de `NODE_ENV` dans `config.ts:11` est `"development"` : si le conteneur démarrait sans `NODE_ENV=production`, toutes ces protections disparaîtraient silencieusement. (`docker-compose.yml:99` fixe bien `NODE_ENV: production` — ✅ Vérifié pour ce déploiement.)

6.5 Limitation de débit

✅ **Vérifié** — `backend/src/middleware/security.ts`

LIMITEUR	FENÊTRE / PLAFOND	CLÉ	NOTE
<code>globalLimiter</code>	60 s / 120 req	<code>sub</code> du JWT, sinon IP réelle	Les admins ne sont pas limités (rôle lu dans le JWT — un token admin volé est illimité)
<code>authLimiter</code>	15 min / 200 échecs	IP réelle (<code>CF-Connecting-IP</code>)	<code>skipSuccessfulRequests: true</code>
<code>registerLimiter</code>	1 h / 100	IP réelle	Plafond volontairement large (CGNAT mauritanien)
<code>submitLimiter</code>	60 s / 30	userId	
<code>browseLimiter</code>	60 s / 15	userId	Alerte email au 3 ^e dépassement
<code>antiSpeedMiddleware</code>	10 s entre deux soumissions	userId	Map en mémoire par worker
<code>antiScrapingMiddleware</code>	80 lectures / 5 min sans action	userId	Basé sur <code>audit_log</code> → partagé entre workers

⚠ **Limite structurelle :** `express-rate-limit` utilise un store **en mémoire**, non partagé entre les 4 workers cluster (et perdu au redémarrage). Les plafonds effectifs sont donc jusqu'à **x4** et remis à zéro à chaque déploiement. Le code le reconnaît pour `antiSpeedMiddleware` (`security.ts:259-262`). ❌ Aucun store Redis.

7. Modèle de données, fichiers stockés et données sensibles

7.1 Tables (54 définies dans backend/sql/)

✓ **Vérfié** — ⚠ `votes`, `votable_phrases` et `season0_contributors` sont utilisées par le code **sans définition dans le dépôt** (cf. §4.4.2). `_season0_reset_done` est une table technique de garde créée par `migration_v23`.

DOMAINE	TABLES
Identité / session	<code>users</code> , <code>profiles</code> , <code>refresh_tokens</code> , <code>otp_codes</code> , <code>password_resets</code> , <code>email_verifications</code> , <code>admin_ip_whitelist</code> , <code>user_events</code>
Corpus	<code>phrases</code> , <code>phrase_locks</code> , <code>phrase_skips</code> , <code>phrase_assignments</code> , <code>tags</code> , <code>contribution_tags</code> , <code>lexicon</code>
Production	<code>contributions</code> , <code>validations</code> , <code>validations_archive</code> , <code>votes</code> , <code>reviews</code> , <code>failed_submissions</code> , <code>eval_void_batch</code>
Média	<code>video_datasets</code> , <code>video_clips</code> , <code>media_ingest_jobs</code> , <code>media_transcriptions</code> , <code>media_locks</code> , <code>media_honeypots</code> , <code>media_honeypot_results</code>
Gamification / compétition	<code>competition_config</code> , <code>point_multipliers</code> , <code>schedule_slots</code> , <code>credit_plans</code> , <code>credit_windows</code> , <code>credit_consumption</code> , <code>credit_usage_hourly</code> , <code>flash_tournaments</code> , <code>flash_tournament_contributions</code> , <code>flash_tournament_validations</code> , <code>flash_tournament_snapshots</code> , <code>flash_tournament_exclusions</code> , <code>referrals</code> , <code>contributor_credits</code>
Communauté	<code>community_cards</code> , <code>community_votes</code> , <code>community_comments</code> , <code>community_card_likes</code> , <code>community_voice_likes</code> , <code>community_voice_endorsements</code>
Notifications	<code>admin_notifications</code> , <code>admin_notification_recipients</code> , <code>app_notifications</code> , <code>broadcast_lists</code>
Traçabilité / divers	<code>audit_log</code> , <code>disclaimer_acceptances</code> , <code>dataset_imports</code> , <code>ai_news_seen</code>

7.2 Champs personnels et sensibles

✓ **Vérfié** — table `users`

CHAMP	SENSIBILITÉ	PROTECTION CONSTATÉE
<code>password_hash</code>	● Critique	bcrypt coût 12
<code>nni</code> (identité nationale mauritanienne)	● Donnée d'identité officielle	Aucun chiffrement au repos , unicité en base, immuable après inscription (trigger)
<code>whatsapp</code>	● Contact direct	Normalisé E.164, immuable, unique
<code>email</code>	●	Synthétisé en <code>wa-<digits>@whatsapp.local</code> si absent
<code>first_name</code> , <code>last_name</code> , <code>birthdate</code>	●	Immuable après inscription

✓ **Vérfié** — table `profiles` (forensique anti-fraude, `migration_v17_anti_fraud.sql`)

CHAMP	SENSIBILITÉ	NOTE
<code>device_fingerprint</code>	● Traçage inter-comptes	SHA-256 canvas + WebGL + écran + TZ + UA (<code>src/lib/fingerprint.ts</code>)
<code>signup_ip</code>	● Donnée personnelle (RGPD)	Indexée pour le clustering
<code>signup_geo</code> (JSONB)	●	Pays, ville, ISP, ASN, flags proxy/hosting
<code>signup_ua</code>	●	Tronqué à 500 caractères
<code>fraud_score</code> , <code>fraud_flags</code>	● Décision automatisée	Rejet automatique d'inscription si score $\geq 0,8$

✓ **Vérifié** — table `contributor_credits` (`migration_v40_contributor_credits.sql`)

● **Point RGPD majeur** : cette table stocke le **nom et le numéro WhatsApp de tierces personnes** (les aidants du contributeur), avec un champ `consent` (`cite` / `anonymous`) saisi **par le contributeur, pas par la personne concernée**. Jusqu'à 60 entrées par utilisateur (`routes/credits.ts:32`). Aucune base légale documentée, aucun mécanisme d'information ou de retrait pour ces tiers.

7.3 Fichiers stockés

✓ **Vérifié** — volume Docker `recordings` monté sur `/data/recordings` (`UPLOAD_DIR`)

CHEMIN	CONTENU	EXPOSITION	TAILLE MAX
<code><uploadDir>/<userId>/</code>	Audio des contributions (<code>.webm</code> / <code>.wav</code>) + un sidcar <code>.json</code> contenant <code>user_id</code> , texte source, traduction, horodatage	JWT + compétition active + URL signée HMAC 10 min (admins exemptés)	10 Mo (<code>MAX_AUDIO_SIZE_MB</code>)
<code><uploadDir>/media/</code>	Clips vidéo/audio découpés + vidéo originale recompressée	URL signée (30 min / 2 h) ou Bearer admin	800 Mo à l'ingestion
<code><uploadDir>/community/</code>	Images, notes vocales, vidéos communautaires + médias de notification	● PUBLIC, sans authentification (<code>server.ts:175-178</code>)	60 Mo (25 Mo notifs)
<code><uploadDir>/failed/</code>	Coffre des soumissions rejetées	Admin	—
<code><uploadDir>/_tmp/</code> , <code>_chunks/</code>	Fichiers temporaires multer / chunks	—	30 Mo/chunk
<code>_dataset_uploads/</code> (ou <code>DATASET_UPLOAD_DIR</code>)	CSV de remplacement de dataset	Admin, <code>mode 0o700</code> , purge > 1 h	500 Mo

● **Nature biométrique des données** : les enregistrements vocaux constituent des **données à caractère personnel de nature biométrique**. Le dépôt ne contient **aucune** politique de conservation, de suppression, d'anonymisation ni de portabilité. ✗ **Absent**

7.4 Traçabilité — audit_log

✓ **Vérifié** — backend/src/utils/audit.ts. Champs : user_id, action, target_type, target_id, details (JSONB), ip_address, created_at.

~40 actions journalisées, dont : register, register_collision, register_rejected_fraud, login, logout, phrase_served, contribution_submitted, validation_submitted, vote_submitted, self_validation_attempt, audio_duplicate_rejected, audio_invalid_magic, audio_quality_rejected, submit_without_lock, refresh_token_reuse_detected, password_reset_*, collusion_auto_detected, scraping_alert_sent, starter_quota_blocked, phrase_reported, flash_tournament_*, tous les changements de configuration admin.

⚠ **audit_log** contient des données personnelles : details embarque nni, whatsapp, email et l'empreinte device à l'inscription (auth.ts:246, auth.ts:344), plus ip_address sur chaque entrée. ✗ Aucune politique de rétention/purge (à la différence de refresh_tokens, purgé toutes les 6 h). La table sert aussi de compteur anti-scraping et de base au plafond quotidien → sa croissance est un **risque de performance et de conformité**.

7.5 Conformité

ÉLÉMENT	STATUT
Politique de confidentialité / mentions légales	✗ Absent
Registre des traitements, base légale, DPO	✗ Absent
Consentement explicite pour l'audio (biométrie)	⚠ Partiel : mécanisme disclaimer_enabled + disclaimer_acceptances versionné (migration_v54_disclaimer.sql), mais désactivé par défaut et son contenu est saisi en base par l'admin
Droit à l'effacement / portabilité	✗ Absent (aucun endpoint de suppression de compte)
Durée de conservation	✗ Absent
Transferts hors UE	⚠ Audio et textes envoyés à OpenAI ; IP envoyées à ip-api.com — aucun DPA documenté
Chiffrement au repos (NNI, audio)	✗ Absent

8. Variables d'environnement, secrets et configurations

8.1 Variables backend

✓ Vérifié — backend/src/config.ts + process.env.*

VARIABLE	REQUISE	DÉFAUT	RÔLE	DANS DOCKER-COMPOSE.YML
NODE_ENV	non	development	● Active/désactive originEnforced + gate X-API-Key	✓ production
PORT	non	4000	Port d'écoute	✓
WEB_CONCURRENCY	non	min(cœurs, 4)	Workers cluster	✓ 4
DB_HOST / DB_PORT / DB_NAME / DB_USER	non	postgres / 5432 / hassaniya / hassaniya	Connexion DB	✓
DB_PASSWORD	oui	—	● Secret DB (\${DB_PASSWORD:?...})	✓
DB_MAX_CONNECTIONS	non	100	Taille du pool par worker	✓ 100
JWT_SECRET	oui	—	● Signature JWT + dérivation de 4 autres secrets	✓ (: ? obligatoire)
JWT_ACCESS_EXPIRES / JWT_REFRESH_EXPIRES	non	3h / 7d	⚠ Lues mais jamais appliquées	✗
FRONTEND_URL	oui	http://localhost:3000	Origine CORS unique	✓
RATE_LIMIT_WINDOW_MS / RATE_LIMIT_MAX_REQUESTS	non	60000 / 60	Limiteur global (plancher 120)	✗
PHRASE_LOCK_DURATION_MS	non	300000	Verrou de phrase	✗
MIN_SUBMISSION_INTERVAL_MS	non	10000	Anti-vitesse	✗
MIN_CONTRIBUTIONS_TARGET	non	3	Cible de couverture	✗
UPLOAD_DIR	non	/data/recordings	Racine des fichiers	✓
MAX_AUDIO_SIZE_MB	non	10	Taille max audio	✗
SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASS	non	smtp.zoho.com / 587	● Secret SMTP ; email désactivé si vide	✓
ALERT_EMAILS	non	""	Destinataires des alertes sécurité	✓
API_KEY_SECRET	non	dérivé de JWT_SECRET	● Secret HMAC de la clé de session	✗
API_KEY	non	—	Ancien nom, utilisé en repli	✓

VARIABLE	REQUISE	DÉFAUT	RÔLE	DANS DOCKER-COMPOSE.YML
AUDIO_URL_SECRET	non	dérivé de JWT_SECRET	● Secret des URL audio signées	✗
OTP_PEPPER	non	= JWT_SECRET	● Poivre des codes OTP	✗
OPENAI_API_KEY	non	—	● Clé OpenAI ; fonctions IA muettes si absente	✗ jamais transmise au conteneur
WAHA_URL / WAHA_SESSION / WAHA_API_KEY	non	http://waha:3000 / default / ""	● Secret WAHA ; WhatsApp désactivé si vide	✓
ADMIN_BOOTSTRAP	non	—	● Contourne la whitelist IP admin	✗
DATASET_UPLOAD_DIR	non	<UPLOAD_DIR>/_dataset_uploads	Répertoire CSV admin	✗
SEED_FLORES	non	false	Seed au boot	✓

8.2 Variables frontend

✓ **Vérifié** — une seule : NEXT_PUBLIC_API_URL , injectée comme ARG au build Docker (Dockerfile:8-9 , docker-compose.yml:135-136). Toute modification exige un **rebuild de l'image frontend**.

8.3 Variables des scripts de déploiement

✓ **Vérifié** — deploy/daily-leaderboard-email.mjs , deploy/daily-leaderboard.mjs : BULK_SMTP_HOST , BULK_SMTP_PORT , BULK_SMTP_USER , BULK_SMTP_PASS , BULK_SMTP_FROM , EMAIL_TEST , plus les SMTP_* standards. deploy/disk-alert.sh:14 lit /opt/hassaniya/.env en clair pour récupérer SMTP_USER / SMTP_PASS et les passe à curl --user (visibles dans la table des process).

8.4 Gestion des secrets — constats

CONSTAT	PREUVE	GRAVITÉ	STATUT
.env.example inexistant alors que le README ordonne cp .env.example .env	README.md:66,90 vs ls	● Moyenne — impossible d'initialiser un environnement sans connaissance tacite	✗ Absent
4 secrets dérivés d'un seul : API_KEY_SECRET , AUDIO_URL_SECRET , OTP_PEPPER et le token dataset retombent tous sur JWT_SECRET , et aucun n'est présent dans docker-compose.yml → en pratique tous dérivés	utils/api-key.ts:7-12 , utils/audio-token.ts:23-28 , config.ts:29 , utils/dataset-token.ts:17	● Élevée — une fuite de JWT_SECRET compromet simultanément : sessions, clés d'API, URL audio signées, codes OTP, tokens d'export dataset, codes anonymes du classement, codes de parrainage	✓ Vérifié
Le code avertit lui-même de cette dérivation	console.warn("[SEC] API_KEY_SECRET not set...")	—	✓ Vérifié

CONSTAT	PREUVE	GRAVITÉ	STATUT
<code>OPENAI_API_KEY</code> absente de <code>docker-compose.yml</code>	<code>docker-compose.yml:98-120</code>	● Fonctionnel : transcription IA, traduction IA et bot de veille silencieusement inactifs en Docker	✓ Vérifié
Aucun gestionnaire de secrets (Vault, Secret Manager, Docker secrets) — tout en variables d'environnement dans un <code>.env</code> sur l'hôte	—	●	✗ Absent
Aucune rotation de secrets documentée	—	●	✗ Absent
Aucun secret en dur trouvé dans le code source	Vérifié par recherche	✓ Bon point	✓ Vérifié
<code>.gitignore</code> couvre <code>.env*</code> , <code>*.sql.gz</code> , <code>/users</code> ("leaked credentials export — sensitive"), <code>*.pem</code> , <code>*.csv</code>	<code>.gitignore:33-63</code>	✓ Bon point ⚠ la ligne <code>/users</code> suggère qu'un export de credentials a existé localement	✓ Vérifié
Mot de passe admin par défaut en clair dans le schéma	<code>init.sql:283-293</code>	● Critique — cf. §11.1	✓ Vérifié

9. Installation, lancement local, build, tests et Docker

9.1 Prérequis

✓ **Vérfié** — `README.md:54-58` : Docker + Docker Compose (voie recommandée), **ou** Node.js ≥ 20 + PostgreSQL 17 local. ⚠ Non documenté dans le README mais **requis** pour le traitement audio/média en local : `ffmpeg` , `ffprobe` , `yt-dlp` , `python3` (présents uniquement dans l'image Docker backend).

9.2 Démarrage Docker complet

✓ **Vérfié** — `README.md:85-97`

```
cp .env.example .env      # ✗ ce fichier n'existe pas dans le dépôt
docker compose up -d --build
curl http://localhost/api/health
docker compose logs -f backend
```

`init.sql` est monté sur `/docker-entrypoint-initdb.d/01-init.sql` et ne s'exécute **qu'au premier démarrage** du volume `pg_data` . Les 72 migrations restent à appliquer à la main.

9.3 Développement local

✓ **Vérfié** — `README.md:101-129`

```
docker compose up -d postgres      # 1. base
cd backend && npm install && npm run dev  # 2. API :4000 (tsx watch)
npm install && npm run dev              # 3. front :3000 (à la racine)
```

9.4 Scripts disponibles

✓ **Vérfié**

FRONTEND (RACINE)	BACKEND (<code>CD BACKEND</code>)
<code>npm run dev</code> — Next.js dev	<code>npm run dev</code> — <code>tsx watch src/server.ts</code>
<code>npm run build</code> — build prod	<code>npm run build</code> — <code>tsc</code> → <code>dist/</code>
<code>npm run start</code> — serveur prod	<code>npm run start</code> — <code>node dist/server.js</code>
<code>npm run lint</code> — ESLint	<code>npm run lint</code> — <code>eslint src/</code>
—	<code>npm run seed</code> — seed FLORES

9.5 Tests

ÉLÉMENT	STATUT
Tests unitaires	✗ Absent
Tests d'intégration / API	✗ Absent
Tests end-to-end	✗ Absent
Framework de test installé (jest, vitest, playwright...)	✗ Absent
Fichiers <code>*.test.*</code> / <code>*.spec.*</code>	✗ Absent (0 fichier)
Script manuel de vérification	⚠ <code>backend/src/scripts/test-v5.ts</code> — script ponctuel, non exécuté par le CI
Pipeline CI	✗ Absent (pas de <code>.github/</code>)
Traces de tests dans le code	✅ <code>community.ts:383</code> filtre les artefacts <code>/_e2e/</code> , ce qui suggère des tests e2e hors dépôt — ⚠ à confirmer

➡ Un dépôt de ~35 000 lignes gérant argent (prix), identité (NNI) et biométrie (voix) sans aucun test automatisé constitue un risque de régression majeur.

9.6 Docker — analyse des images

✅ Vérifié

Points forts :

- Build **multi-stage** pour les deux images (`Dockerfile` , `backend/Dockerfile`).
- **Utilisateur non-root** dans les deux runners (uid/gid 1001 : `nextjs` , `api`).
- `postgres` et `pgbouncer` n'exposent **aucun port sur l'hôte** (`expose` seulement).
- Limites mémoire déclarées pour `postgres` (2 Go), `backend` (1 Go), `waha` (768 Mo).
- `healthcheck` sur `postgres` et `pgbouncer` ; Caddy sonde `/api/health` toutes les 10 s.
- `.dockerignore` exclut `node_modules` , `dist` , `.env` , `*.log` .

Points faibles :

CONSTAT	RÉF.	STATUT
<code>npm install</code> (lockfile ignoré) au lieu de <code>npm ci</code>	<code>Dockerfile:5</code> , <code>backend/Dockerfile:5</code>	✅ Vérifié
Les devDependencies sont copiées dans le runner backend (<code>COPY --from=builder /app/node_modules</code>) — pas de <code>npm prune --omit=dev</code>	<code>backend/Dockerfile:26</code>	✅ Vérifié
<code>yt-dlp</code> téléchargé depuis GitHub <code>latest</code> au build, sans vérification de somme de contrôle	<code>backend/Dockerfile:17-18</code>	✅ Vérifié
Images de base non épinglées par digest (<code>node:22-alpine</code> , <code>caddy:2-alpine</code>), deux services en <code>:latest</code>	<code>docker-compose.yml:58,211</code>	✅ Vérifié
Aucun <code>read_only</code> , <code>cap_drop</code> , <code>security_opt</code> , <code>no-new-privileges</code> ni <code>user:</code> dans le compose	<code>docker-compose.yml</code>	✗ Absent

CONSTAT	RÉF.	STATUT
Aucune configuration <code>logging</code> (driver/rotation) → logs <code>json-file</code> non bornés	<code>docker-compose.yml</code>	✗ Absent — risque de saturation disque
Aucune limite CPU/mémoire sur <code>caddy</code> et <code>frontend</code>	<code>docker-compose.yml</code>	⚠
<code>deploy.sh:43</code> exécute <code>docker compose down</code> — le README interdit explicitement cette commande en production (<code>README.md:175</code>) : incohérence dangereuse entre la doc et le script	✓ Vérifié	
Scan de vulnérabilités d'image (Trivy, Gype...)	—	✗ Absent

10. Logs, gestion des erreurs et observabilité

10.1 Journalisation applicative

✓ **Vérifié** — exclusivement `console.log` / `console.warn` / `console.error`, avec des préfixes conventionnels : `[CLUSTER]`, `[DB]`, `[API]`, `[CORS]`, `[SECURITY]`, `[ERROR]`, `[AUDIT]`, `[PURGE]`, `[NOTIF]`, `[AUTO-STATUS]`, `[AUTO-ACTIVATION]`, `[COLLUSION]`, `[NEWS]`, `[WAHA]`, `[EMAIL]`, `[SEC]`, `[ADMIN-DENY]`, `[ADMIN-BOOTSTRAP]`, `[media-ingest]`, `[recut]`, `[ingest-resume]`, `[AUDIO]`, `[disk-alert]`.

ÉLÉMENT	STATUT
Logger structuré (pino, winston) / logs JSON	✗ Absent
Corrélation de requêtes (<code>request-id</code> , <code>trace-id</code>)	✗ Absent
Niveaux de log configurables	✗ Absent
Agrégation centralisée (ELK, Loki, Cloud Logging)	✗ Absent — uniquement <code>docker compose logs</code>
Rotation / rétention des logs	✗ Absent
Journal d'accès HTTP (Caddy)	✗ Absent — aucune directive <code>log</code> dans le <code>Caddyfile</code>
Rédaction des données personnelles dans les logs	✗ Absent — cf. §10.4

10.2 Gestion des erreurs

✓ **Vérifié** — plusieurs bonnes pratiques :

- **Handler d'erreur global** (`server.ts:375-390`) : le détail technique va aux logs, le client reçoit `{ error: "<message FR générique>", code: "TEMP_ERROR" }` en 500 — **aucune fuite de stack trace ni de message SQL**.
 - **Codes d'erreur métier normalisés**, exploités par le client : `TOKEN_EXPIRED`, `SESSION_REVOKED`, `MISSING_REQUESTED_BY`, `COMPETITION_NOT_ACTIVE`, `COMPETITION_ENDED`, `NOT_ACTIVATED`, `NOT_ACTIVATED_YET`, `EMAIL_NOT_VERIFIED`, `ONBOARDING_REQUIRED`, `PAUSED`, `CLOSED`, `CREDIT_EXHAUSTED`, `CREDIT_PAUSED`, `CREDIT_BLOCKED`, `LANG_CLOSED`, `EVAL_LOCKED`, `AUDIO_DUPLICATE`, `AUDIO_QUALITY_REJECTED`, `AUDIO_SIG_INVALID`, `ALREADY_VALIDATED`, `ALREADY_DECIDED`, `CONTRIBUTION_GONE`, `CLIP_GONE`, `TOO_FAST`, `DISCLAIMER_REQUIRED`, `STARTER_QUOTA_BLOCKED`, `COMMUNITY_BLOCKED`, `TOP_N_ONLY`, `PROPOSE_COOLDOWN`, `REGISTRATION_CLOSED`, `INVITE_REQUIRED`, `INVITE_INVALID`, `COUNTRY_NOT_ALLOWED`, `ADMIN_IP_NOT_WHITELISTED`.
 - **Anti-perte de travail** : tout 5xx est traité comme *retryable* par l'outbox client ; toute soumission refusée est archivée dans `failed_submissions`.
 - **Dégradation gracieuse** : `try/catch` autour de la géo-IP, du lexique, des tables manquantes (`42P01` / `42703`), de FFmpeg (le code laisse passer la contribution si FFmpeg échoue — `phrases.ts:525-530`).
 - **Résilience** : redémarrage automatique des workers cluster, `retries: 10` sur la connexion DB au boot, reprise des ingestions orphelines.
- ⚠ **Réserve** : `console.error("[ERROR]", err.message)` ne journalise **que le message**, pas la stack — le diagnostic post-mortem est appauvri.

10.3 Observabilité

CAPACITÉ	IMPLÉMENTATION	STATUT
Endpoint de santé	<code>GET /api/health</code> (vérifie la DB, 200/503)	✓ Vérifié
Sonde active	Caddy <code>health_uri /api/health</code> , <code>health_interval 10s</code>	✓ Vérifié
Healthchecks conteneurs	<code>postgres</code> , <code>pgbouncer</code>	✓ Vérifié
Healthcheck <code>backend</code> / <code>frontend</code> / <code>caddy</code>	—	✗ Absent
Métriques (Prometheus, <code>/metrics</code>)	—	✗ Absent
Tracing distribué (OpenTelemetry)	—	✗ Absent
APM / suivi d'erreurs (Sentry...)	—	✗ Absent
Uptime monitoring externe	—	✗ Absent
Alerting sécurité	Emails via <code>ALERT_EMAILS</code> : scraping, abus de rate-limit, inscription suspecte, collusion ; cooldown 5 min/type/utilisateur	✓ Vérifié
Alerting capacité	<code>deploy/disk-alert.sh</code> — cron horaire, seuil 70 %, dédup 24 h	✓ Vérifié
Tableaux de bord internes	Console admin : <code>/admin/realtime</code> , <code>/admin/x/security</code> , <code>/admin/productivity</code> , <code>/admin/throughput</code> , <code>collusion-report</code> , <code>competition-integrity</code> , <code>fairness-dashboard</code> , <code>credit-tracking</code>	✓ Vérifié
Rapport quotidien	<code>deploy/daily-leaderboard*.mjs</code> + <code>top10_email.html</code> (cron)	✓ Vérifié

10.4 Données personnelles dans les journaux

● ✓ **Vérifié** — les alertes de sécurité par email contiennent en clair :

```
auth.ts:375 → username, email, nni, whatsapp, IP, pays, ville, ISP, flags proxy/hosting
alert.ts:39 → username, prénom, nom, email, rôle
```

Et `audit_log.details` embarque `nni`, `whatsapp`, `email`, `fingerprint` à l'inscription. ✗ Aucune redaction, aucune durée de conservation. **Non conforme au principe de minimisation.**

11. Risques de sécurité (grille OWASP)

Chaque risque est noté **Gravité** (Critique / Élevée / Moyenne / Faible) au regard du contexte : compétition dotée de prix en numéraire, données d'identité nationale, corpus de grande valeur, biométrie vocale.

11.1 ● CRITIQUE — Compte administrateur par défaut avec mot de passe en clair

OWASP	A07 Identification & Authentication Failures / A05 Security Misconfiguration
Preuve	<code>backend/sql/init.sql:283-293</code>
Statut	✅ Vérifié

`init.sql` insère un compte `admin` (email `admin@adst.io`, rôle `admin`, `email_verified = true`) avec un **mot de passe littéral en clair dans le fichier de schéma** et un **UUID fixe et connu**. Ce fichier est versionné dans le dépôt.

Scénario d'exploitation : toute personne ayant lu le dépôt (ou la documentation publique de la stack) connaît l'identifiant et le mot de passe. Sur tout environnement fraîchement initialisé où le mot de passe n'a pas été changé, l'authentification admin réussit du premier coup.

Atténuation en place : `adminMiddleware` exige en plus une **IP whitelistée** et échoue *fail-closed* si la table est vide — sauf si `ADMIN_BOOTSTRAP=true` est laissé positionné (le code lui-même avertit : `[ADMIN-BOOTSTRAP] user=... accessing admin without whitelist`).

Correctifs 💡 :

- Retirer le `INSERT` du schéma ; créer l'admin via un script hors dépôt à mot de passe généré aléatoirement, ou via une commande d'amorçage exigeant `ADMIN_INITIAL_PASSWORD`.
- Sur les environnements existants : ⚠️ à **confirmer** que le mot de passe a été changé et que `ADMIN_BOOTSTRAP` n'est pas positionné.
- Ajouter une contrainte forçant le changement au premier login.

11.2 ● ÉLEVÉE — Mono-secret : `JWT_SECRET` gouverne 7 mécanismes cryptographiques

OWASP	A02 Cryptographic Failures
Preuves	<code>utils/api-key.ts:7-12</code> , <code>utils/audio-token.ts:23-28</code> , <code>config.ts:29</code> , <code>utils/dataset-token.ts:17</code> , <code>utils/anon.ts:13,33,65</code>
Statut	✅ Vérifié

En l'absence de `API_KEY_SECRET`, `AUDIO_URL_SECRET` et `OTP_PEPPER` — **aucun des trois n'est transmis au conteneur backend par `docker-compose.yml`** — tous dérivent déterministiquement de `JWT_SECRET`, qui sert par ailleurs directement au token d'export dataset, aux codes anonymes du classement et aux codes de parrainage.

Impact d'une fuite unique de `JWT_SECRET` : forge de tokens d'accès et de refresh pour n'importe quel compte (y compris admin), forge de clés `X-API-Key`, forge d'URL audio signées (**exfiltration de tout le corpus audio**), calcul des codes OTP, forge du token d'export dataset, **dé-anonymisation du classement** et des codes de parrainage.

Correctifs 💡 : générer 4 secrets indépendants (`openssl rand -hex 64`), les injecter explicitement dans le compose, faire échouer le démarrage si l'un manque en production, documenter une procédure de rotation.

11.3 ● ÉLEVÉE — PgBouncer en `AUTH_TYPE=trust` sur le réseau partagé

OWASP	A05 Security Misconfiguration / A01 Broken Access Control
Preuves	<code>docker-compose.yml:78</code> , <code>docker-compose.yml:101-102</code>
Statut	✅ Vérifié

`pgbouncer` accepte les connexions **sans authentification** (`AUTH_TYPE: trust`) et relaie vers PostgreSQL avec les identifiants applicatifs complets. Il est joignable par **tout conteneur du réseau `internal`** , y compris `waha` — une image tierce (`devlikeapro/waha:latest`) qui pilote un vrai compte WhatsApp et communique avec Internet.

Aggravant : le backend est configuré sur `DB_HOST: postgres` et **n'utilise jamais PgBouncer**. Le service constitue donc une **surface d'attaque sans contrepartie fonctionnelle**, tout en laissant croire que le pooling est en place (400 connexions théoriques du pool applicatif face à `max_connections=150`).

Correctifs 💡 : soit supprimer `pgbouncer` du compose, soit l'activer réellement (`DB_HOST: pgbouncer`) avec `AUTH_TYPE: scram-sha-256` et un `userlist.txt` . Dans les deux cas, isoler `waha` sur un réseau Docker distinct.

11.4 ● ÉLEVÉE — Token d'export du dataset non lié à un utilisateur

OWASP	A01 Broken Access Control
Preuves	<code>utils/dataset-token.ts:10-20</code> , <code>routes/users.ts:661-668</code> , <code>server.ts:143</code> , <code>server.ts:364-367</code>
Statut	✅ Vérifié

Le token est un HMAC calculé sur `lang:audio:exp` **uniquement** — ni `userId` , ni IP, ni usage unique. L'endpoint `GET /api/dataset-zip` est explicitement **exclu du gate `X-API-Key`** et **n'est pas couvert par `originEnforced`** , afin de permettre un téléchargement par navigation directe.

Scénario : l'URL complète (avec `sig`) transite dans l'historique du navigateur, les journaux du proxy, éventuellement un `Referer` . Quiconque l'obtient dans les **5 minutes** télécharge **l'intégralité du corpus validé** : `metadata.csv` (phrase source, traduction hassaniya, scores) **plus tous les fichiers audio**. C'est l'actif principal du projet.

Correctifs 💡 : inclure `userId` dans la charge signée et le vérifier ; réduire le TTL ; rendre le token à usage unique (nonce en base) ; journaliser chaque téléchargement dans `audit_log` .

11.5 ● ÉLEVÉE — Codes de réinitialisation générés avec `Math.random()`

OWASP	A02 Cryptographic Failures / A07
Preuves	<code>auth.ts:805</code> (reset mot de passe), <code>auth.ts:729</code> (vérification email)
Statut	✅ Vérifié

```
const code = String(Math.floor(100000 + Math.random() * 900000));
const codeHash = crypto.createHash("sha256").update(code).digest("hex");
```

`Math.random()` n'est **pas cryptographiquement sûr** : son état interne est prédictible à partir de sorties observées. De plus, le hachage est un **SHA-256 sans poivre** (1 million de pré-calculs suffisent en cas de fuite de la base), alors que le module OTP fait correctement les deux : `crypto.randomInt` + HMAC poivré (`utils/otp.ts:16-18`).

Correctifs 💡 : remplacer par `crypto.randomInt(0, 1_000_000)` et par `crypto.createHmac("sha256", config.otpPepper)` dans les deux chemins, à l'image de `utils/otp.ts`.

11.6 ● ÉLEVÉE — Politique de mot de passe faible, sans verrouillage de compte

OWASP	A07 Identification & Authentication Failures
Preuves	<code>auth.ts:126</code> (<code>z.string().min(6)</code>), <code>auth.ts:833</code> , <code>middleware/security.ts:80-88</code>
Statut	✅ Vérifié

- Minimum **6 caractères**, aucune exigence de complexité, aucune vérification contre les mots de passe compromis, aucun blocage des mots de passe triviaux.
- **Aucun verrouillage par compte** après N échecs.
- Le seul garde-fou est `authLimiter` : 200 **échecs** / 15 min **par IP réelle**. Or le code documente lui-même que les opérateurs mauritaniens placent « des MILLIERS d'abonnés derrière une poignée d'IP CGNAT » (`security.ts:96-99`) — le plafond est donc volontairement large.

Scénario : depuis quelques IP distinctes, un attaquant peut tester des mots de passe faibles sur de nombreux comptes (*credential stuffing* / *password spraying*) sans jamais déclencher de blocage de compte. Dans une compétition dotée de prix, la prise de contrôle d'un compte de tête de classement a une valeur directe.

Correctifs 💡 : minimum 10–12 caractères, contrôle contre une liste de mots de passe compromis, **compteur d'échecs par compte** avec temporisation exponentielle, alerte `suspicious_login` (le type d'alerte existe déjà dans `alert.ts:14` mais n'est jamais émis).

11.7 ● MOYENNE-ÉLEVÉE — Un seul utilisateur peut désactiver le corpus

OWASP	A01 Broken Access Control / A04 Insecure Design
Preuve	<code>routes/phrases.ts:224-248</code>
Statut	✅ Vérifié

`POST /api/phrases/report` exécute directement :

```
UPDATE phrases SET is_active = false WHERE id = $1 AND is_active = true
```

sans seuil, sans quorum, sans revue. Le commentaire du code l'assume (« hidden IMMEDIATELY »). Le seul frein est `globalLimiter` (120 req/min/utilisateur).

Scénario : un compte authentifié quelconque itère sur les identifiants de phrases et désactive ~120 phrases/minute, soit **plus de 170 000 par 24 h** — le corpus servi devient vide et la compétition s'arrête. C'est un déni de service applicatif à faible coût.

Correctifs 💡 : exiger N signalements distincts (ou une confirmation admin) avant désactivation ; limiter à quelques signalements par utilisateur et par jour ; alerter au-delà d'un seuil.

11.8 🟠 MOYENNE-ÉLEVÉE — Blocage automatique de compte par le quota "starter"

OWASP	A04 Insecure Design (disponibilité / équité)
Preuve	<code>routes/phrases.ts:40-91</code>
Statut	✅ Vérifié

Si un utilisateur n'a pas envoyé **10 contributions dans les 30 minutes** suivant sa première requête, le serveur exécute `UPDATE users SET is_active = false` — le compte est **désactivé**, ce qui invalide immédiatement toutes ses sessions (`middleware/auth.ts:107-110`).

Risques : une coupure réseau, un micro défaillant ou une simple lenteur suffisent à bannir un participant légitime ; il n'existe **aucun endpoint de réactivation automatique** (seul un admin peut rétablir via `/admin/x/users/:id/activate`). Dans une compétition avec prix, cela expose à une contestation d'équité.

Correctifs 💡 : remplacer la désactivation du compte par un simple statut de quarantaine réversible, informer l'utilisateur en amont, et rendre le quota configurable (actuellement en dur : `STARTER_QUOTA_MINUTES = 30`, `STARTER_QUOTA_CONTRIBUTIONS = 10`).

11.9 🟠 MOYENNE — Répertoire média communautaire public, sans validation de contenu

OWASP	A01 / A04 — Upload dangereux
Preuves	<code>server.ts:170-178</code> , <code>routes/community.ts:96-117</code> , <code>routes/users.ts:2088-2099</code>
Statut	✅ Vérifié

`/recordings/community/*` est servi **sans aucune authentification** (le code l'assume : « Community feed media [...] is public: unguessable UUID filenames, no competition gate or signed URL »), avec `Cross-Origin-Resource-Policy: cross-origin`.

Les uploads correspondants (jusqu'à **60 Mo**, 25 Mo pour les médias de notification) :

- n'ont **aucun** `fileFilter` `multer` ;
- déterminent l'extension à partir du **MIME déclaré par le client** via une table `EXT` (`community.ts:100-104`) ;
- ne font **aucune validation magic-bytes** (contrairement à l'audio des contributions, `phrases.ts:366-387`) ;
- ne sont soumis à **aucun antivirus** ni quota par utilisateur.

Atténuations réelles : l'extension est contrainte par la table de correspondance, et `express.static` pose `X-Content-Type-Options: nosniff` (`server.ts:252-255`) — un fichier HTML déposé en `.png` sera servi en `image/png` sans être exécuté. Le **XSS stocké est donc peu probable**.

Risque résiduel réel : hébergement de contenu arbitraire sur le domaine du projet (distribution de malware, contenu illicite, abus de bande passante), et exposition publique permanente de notes vocales d'utilisateurs.

Correctifs 💡 : validation magic-bytes, `fileFilter` explicite, ré-encodage systématique des images/audio, `Content-Disposition: attachment` pour les types non prévisualisables, quota par utilisateur, purge programmée.

11.10 🟠 MOYENNE — CSP permissive (`unsafe-inline` + `unsafe-eval`)

OWASP	A03 Injection (XSS)
Preuve	<code>Caddyfile:12</code>
Statut	✅ Vérifié

```
script-src 'self' 'unsafe-inline' 'unsafe-eval'
```

Ces deux directives **annulent l'essentiel** de la protection anti-XSS de la CSP. Le reste de la politique est bon (`default-src 'self'` , `frame-ancestors 'none'` , `object-src` implicite, `connect-src` restreint).

⚠ De plus, la CSP n'est définie **que pour** `elson.adst.ai` ; le bloc `api.elson.adst.ai` (`Caddyfile:54-87`) ne porte ni CSP, ni `Permissions-Policy` , alors qu'il sert des fichiers média téléversés.

Atténuations réelles : React échappe par défaut ; un seul `dangerouslySetInnerHTML` manipule des données utilisateur (`HassaniyaInput.tsx:114`) et applique un échappement manuel de `&` , `<` , `>` sur des tokens filtrés en arabe — la surface est étroite. Un second usage (`layout.tsx:50`) est une constante statique.

Correctifs 💡 : passer à une CSP à **nonces** (supportée par Next.js), supprimer `unsafe-eval` , dupliquer les en-têtes de sécurité sur le bloc API.

11.11 🟠 MOYENNE — Sauvegardes non chiffrées, non externalisées, non testées

OWASP	A08 Software & Data Integrity Failures
Preuve	<code>docker-compose.yml:164-207</code> , <code>deploy/setup-server.sh:106-112</code>
Statut	✅ Vérifié

Le service `pg-backup` produit `db_<date>.sql.gz` + `audio_<date>.tar.gz` dans un **volume Docker du même hôte**, avec rétention de **7 dumps DB / 3 archives audio**.

MANQUE	IMPACT
Aucune copie hors site / hors hôte	Perte de l'hôte = perte des sauvegardes et des données
Aucun chiffrement au repos	Un dump contient tous les NNI, WhatsApp, emails, hashes
Aucun test de restauration documenté	RTO/RPO inconnus
Aucun contrôle d'intégrité (checksum)	Corruption silencieuse possible

MANQUE	IMPACT
<code>tar czf</code> de l'ensemble des audios à chaque exécution (le commentaire annonce « incremental » — ce n'est pas le cas)	Coût disque et temps croissants

11.12 ● MOYENNE — Absence de tests et de CI

OWASP	A08 / A05
Statut	✗ Absent

Aucun test, aucun CI, aucun scan de dépendances ni d'image, `npm install` non déterministe, 72 migrations manuelles. Sur une base de 35 000 lignes portant des règles anti-fraude et un barème de points monétisable, chaque déploiement est un pari.

11.13 ● MOYENNE — Fuites d'informations sur les endpoints ouverts

ENDPOINT	DONNÉE EXPOSÉE	RÉF.	STATUT
<code>GET /api/public/stats</code> (sans auth)	Jusqu'à 300 pseudonymes d'utilisateurs connectés dans les 5 dernières minutes + liste complète des contributeurs "saison 0" (pseudo + nombre de contributions)	<code>server.ts:285-302</code>	✓ Vérifié
<code>GET /api/community/mention-search?q=</code> (authentifié)	Énumération des pseudonymes par préfixe (<code>LIKE \$1</code> avec <code>q + "%"</code> , 8 résultats)	<code>community.ts:247-255</code>	✓ Vérifié
<code>GET /api/community/</code> (authentifié)	Pseudonyme et vote de chaque votant, liste des membres en ligne	<code>community.ts:161,182-186</code>	✓ Vérifié
<code>GET /api/users/leaderboard</code>	Correctement whitelisté : seuls les champs affichés sortent ; pseudonyme remplacé par un code HMAC rotatif toutes les 2 h (sauf si l'admin désactive l'anonymat)	<code>users.ts:1088-1106</code>	✓ Bon point

Contrepartie : l'anti-énumération est **excellente** sur l'inscription (409 générique unique quel que soit le champ en collision, `auth.ts:253-278`) et sur le mot de passe oublié (réponse générique + délai constant de 500 ms, `auth.ts:783-792`).

11.14 ● MOYENNE — Géolocalisation IP en HTTP non chiffré

OWASP	A02 / A08
Preuve	<code>services/geoip.ts:40-43</code> — <code>http://ip-api.com/json/<ip>?fields=...</code>
Statut	✓ Vérifié

À chaque inscription, l'**IP de l'utilisateur est transmise en clair** à un tiers, et la réponse (`proxy` , `hosting` , `countryCode`) **alimente la décision automatisée de rejet** (`fraud.ts:130-146` , seuil de rejet 0,8). Un attaquant en

position d'interception peut falsifier la réponse pour influencer le score, dans un sens comme dans l'autre.

Correctifs 💡 : passer à l'endpoint HTTPS du fournisseur (offre payante) ou à une base locale (MaxMind GeoLite2), ne jamais faire dépendre une décision de sécurité d'un canal non authentifié.

11.15 🟡 MOYENNE — Surface de déni de service

VECTEUR	DÉTAIL	RÉF.	STATUT
Limiteurs non partagés	<code>express-rate-limit</code> en mémoire × 4 workers → plafonds effectifs ×4, remis à zéro à chaque redéploiement	<code>middleware/security.ts</code>	✅ Vérifié
Uploads volumineux	800 Mo (média), 500 Mo (CSV dataset), 60 Mo (communauté), 30 Mo/chunk — aucun quota par utilisateur, aucune vérification d'espace disque avant écriture	<code>media.ts:41</code> , <code>users.ts:2648</code> , <code>community.ts:98</code>	✅ Vérifié
Ré-encodage FFmpeg	Timeouts de 300 s à 1800 s par job d'ingestion, <code>maxBuffer</code> 64 Mo, jusqu'à 300 segments — saturation CPU possible (admin uniquement)	<code>services/media-ingest.ts:96,176</code>	✅ Vérifié
Agrégation lexicale	<code>SELECT ... FROM contributions LIMIT 300000</code> + toutes les <code>media_transcriptions</code> en mémoire pour bâtir le lexique (cache 1 h, mais coût de démarrage à froid par worker)	<code>media.ts:410-434</code>	✅ Vérifié
Boucles HMAC en O(n)	<code>resolveReferralCode</code> charge tous les utilisateurs actifs et calcule un HMAC par utilisateur à chaque inscription avec code de parrainage ; <code>resolveLeaderboardCode</code> fait 84 × n HMAC (admin)	<code>auth.ts:180,299</code> , <code>utils/anon.ts:46-55</code>	✅ Vérifié
<code>audit_log</code> sans purge	Table interrogée à chaud (anti-scraping toutes les 8 s/utilisateur) et jamais purgée	<code>security.ts:186-193</code>	✅ Vérifié
Logs conteneurs non bornés	Aucune configuration <code>logging</code> dans le compose	<code>docker-compose.yml</code>	❌ Absent
Alerte capacité	<code>disk-alert.sh</code> (seuil 70 %) — détection, pas prévention	<code>deploy/disk-alert.sh</code>	✅ Vérifié
WAF / protection DDoS applicative	Référencée dans les commentaires (Cloudflare) mais absente du dépôt	—	⚠️ À confirmer

11.16 ✅ Injections — aucune vulnérabilité trouvée

TYPE	CONSTAT	STATUT
SQL	100 % des requêtes sont paramétrées via <code>pool.query(text, params)</code> . Les seuls fragments dynamiques sont issus de listes blanches fermées : nom de vue <code>whisper_*</code> (<code>phrases.ts:681-682</code>), noms de tables littéraux (<code>community.ts:419-428</code>), clauses <code>ORDER BY</code> issues d'objets de correspondance (<code>users.ts:2718-2722</code> , <code>community.ts:361-365</code>), constantes numériques d'intervalle (<code>security.ts:191</code>). Les filtres admin construisent des <code>\$n</code> incrémentaux, jamais de concaténation de valeurs.	✅ Aucune injection SQL identifiée
Commandes OS	<code>execFile</code> avec tableaux d'arguments partout (<code>ffmpeg</code> , <code>ffprobe</code> , <code>yt-dlp</code>) — jamais de shell , jamais d'interpolation.	✅ Aucune injection de commande

TYPE	CONSTAT	STATUT
SSRF	<code>POST /api/media/ingest</code> passe une URL fournie par un admin à <code>yt-dlp</code> , validée seulement par <code>/^https?:\/\/i</code> (<code>media.ts:127</code>). Un admin peut donc faire émettre des requêtes vers le réseau interne. Le bot de veille récupère des flux RSS et des URL extraites d'une réponse LLM.	● Faible (admin uniquement)
Traversée de chemin	Bonnes gardes <code>path.resolve(...).startsWith(root)</code> dans <code>media-ingest.ts:231-236,250-253</code> et <code>community.ts:547-549</code> . ⚠ Gardes manquantes dans <code>dataset-export.ts:72-74</code> , <code>reviews.ts:269</code> , <code>media.ts:267,514</code> — mais les valeurs viennent de la base (écrites par le serveur), non directement de l'utilisateur.	● Faible — 💡 ajouter la garde par cohérence
XSS	React échappe par défaut ; 2 <code>dangerouslySetInnerHTML</code> seulement (§11.10) ; en-têtes <code>nosniff</code> , <code>X-Frame-Options: DENY</code> , <code>X-XSS-Protection</code> , <code>frame-ancestors 'none'</code> ; les templates email échappent les entrées utilisateur (<code>email.ts:20-27</code>) et rejettent les CRLF dans les sujets (anti-injection d'en-tête SMTP , <code>email.ts:30-32</code>) ; les couleurs CSS communautaires sont validées par regex hexadécimale (<code>community.ts:137-141</code>) ; les URL sont filtrées par <code>cleanUrl</code> (<code>community.ts:85-91</code>).	● Faible
NoSQL / LDAP / XXE / désérialisation	Aucun composant concerné (pas de MongoDB, pas de parseur XML, pas de désérialisation d'objets).	✅ Non applicable

11.17 ● Fraude et intégrité de la compétition — dispositif substantiel

✅ **Vérfié** — c'est le domaine le plus mûr du projet :

CONTRÔLE	IMPLÉMENTATION
Score de fraude à l'inscription	14 signaux (NNI/téléphone séquentiels ou répétés, pseudonyme suspect, email jetable — 35 domaines, UA de bot, empreinte absente, proxy/Tor, datacenter, pays ≠ MR) → rejet ≥0,8, alerte ≥0,4 (<code>utils/fraud.ts</code>)
Détection de multi-comptes	Clustering par <code>device_fingerprint</code> et <code>signup_ip</code> (<code>users.ts:3706-3759</code> , vue <code>signup_forensics</code>)
Détection de collusion	Worker horaire : paire validateur→contributeur avec ≥5 validations à ≥4,5/5 sur 7 jours → +30 <code>fraud_score</code> sur les deux comptes + alerte email (<code>server.ts:614-685</code>) ; vue <code>reciprocity_pairs</code>
Anti-auto-validation	Blocage + journalisation <code>self_validation_attempt</code> (<code>validate.ts:334-345</code>)
Exclusion bidirectionnelle	Un utilisateur ne peut jamais à la fois traduire et évaluer la même phrase (dans les deux sens, verrous inclus) (<code>validate.ts:181-192</code> , <code>phrase-pipeline.ts:151-154</code>)
Déduplication audio	SHA-256 par utilisateur — bloque « un enregistrement → cent phrases » (<code>phrases.ts:389-418</code>)
Rejet du silence	FFmpeg : pic < -35 dB ou durée < 1,5 s → 422 (<code>audio-processor.ts:72-81</code>)
Honeypots	Faux brouillon IA servi ~10 % du temps ; réponse identique pour ne pas prévenir le tricheur ; compteurs <code>hp_seen</code> / <code>hp_failed</code> (<code>media.ts:488-547</code>)
Anonymisation des évaluateurs	Code HMAC opaque, réservé aux admins ; classement à codes rotatifs toutes les 2 h , réversibles par l'admin seul (<code>utils/anon.ts</code>)
Anti-scraping	80 lectures/5 min sans action → 429 + alerte dédoublée 6 h, basé sur <code>audit_log</code> (partagé entre workers) (<code>security.ts:204-242</code>)
Anti-exfiltration audio	URL signées HMAC 10 min liées à l'utilisateur ; audio masqué aux comptes de niveau <2 ; audit échantillonné 1/10 des accès (<code>server.ts:239-244</code>)

CONTRÔLE	IMPLÉMENTATION
Quarantaine / disqualification / annulation d'évaluations	<code>quarantined</code> , <code>disqualified</code> (avec sauvegarde des points), <code>eval_void_batch</code> + simulation avant application
Immuabilité de l'identité	Trigger <code>protect_identity_fields</code>
Traçabilité	~40 actions journalisées dans <code>audit_log</code>

11.18 Synthèse — matrice des risques

#	RISQUE	OWASP	GRAVITÉ	STATUT
11.1	Compte admin par défaut, mot de passe en clair dans <code>init.sql</code>	A07/A05	● Critique	✓ Vérifié
11.2	<code>JWT_SECRET</code> unique dérivant 7 mécanismes crypto	A02	● Élevée	✓ Vérifié
11.3	PgBouncer <code>AUTH_TYPE=trust</code> (et inutilisé) sur réseau partagé avec WAHA	A05/A01	● Élevée	✓ Vérifié
11.4	Token dataset ZIP non lié à un utilisateur → exfiltration du corpus	A01	● Élevée	✓ Vérifié
11.5	Codes de reset via <code>Math.random()</code> , hash non poivré	A02/A07	● Élevée	✓ Vérifié
11.6	Mot de passe ≥6 car., pas de verrouillage de compte	A07	● Élevée	✓ Vérifié
11.7	<code>POST /phrases/report</code> désactive une phrase sans quorum → DoS du corpus	A01/A04	● Moy.- Élevée	✓ Vérifié
11.8	Quota starter désactivant automatiquement les comptes	A04	● Moy.- Élevée	✓ Vérifié
11.9	Média communautaire public, upload sans validation de contenu	A01/A04	● Moyenne	✓ Vérifié
11.10	CSP <code>unsafe-inline</code> / <code>unsafe-eval</code> , absente du domaine API	A03	● Moyenne	✓ Vérifié
11.11	Sauvegardes locales, non chiffrées, non testées	A08	● Moyenne	✓ Vérifié
11.12	Aucun test, aucun CI, aucun scan de dépendances	A08/A05	● Moyenne	✗ Absent
11.13	Fuites de pseudonymes (stats publiques, mention-search)	A01	● Moyenne	✓ Vérifié
11.14	Géo-IP en HTTP alimentant une décision de sécurité	A02/A08	● Moyenne	✓ Vérifié
11.15	DoS : limiteurs par worker, uploads massifs, <code>audit_log</code> sans purge, logs non bornés	A04	● Moyenne	✓ Vérifié

#	RISQUE	OWASP	GRAVITÉ	STATUT
—	Dérive de schéma : 72 migrations manuelles, 10 numéros en doublon, 3 objets DB utilisés sans définition versionnée (<code>votes</code> , <code>votable_phrases</code> , <code>season0_contributors</code>)	A05/A08	● Élevée	✓ Vérifié
—	Dépendance <code>ipaddr.js</code> non déclarée (risque de verrouillage admin total)	A06	● Moyenne	✓ Vérifié
—	PII (NNI, WhatsApp, IP) dans les emails d'alerte et <code>audit_log</code> , sans rétention	Conformité	● Moyenne	✓ Vérifié
—	PII de tiers dans <code>contributor_credits</code> sans base légale	Conformité	● Moyenne	✓ Vérifié
—	Aucune politique de confidentialité / conservation / effacement (audio = biométrie)	Conformité	● Moyenne	✗ Absent
—	Injectons SQL / commandes OS	A03	✓ Aucune trouvée	✓ Vérifié
—	CSRF	A01	● Faible (4 couches)	✓ Vérifié

12. Mesures existantes, éléments manquants et recommandations prioritaires

12.1 Mesures de sécurité déjà en place — Vérifié

Authentification / sessions

- bcrypt coût 12 ; JWT HS256 avec `algorithms` explicite ; rôle **toujours relu en base** ; refresh token `httpOnly` `sameSite=strict` haché en base avec rotation et détection de réutilisation ; **session unique par compte** avec révocation cluster-wide ; clé de session HMAC 1 h ; OTP WhatsApp poivré à comparaison constante ; anti-énumération sur inscription et mot de passe oublié ; `bcrypt` factice pour égaliser les temps de réponse.

Contrôle d'accès

- `adminMiddleware` = rôle + **whitelist IP fail-closed** avec support CIDR IPv6, lisant `CF-Connecting-IP` ; ~165 endpoints admin protégés ; gates métier empilés (email, onboarding, compétition, planning, crédit, quota, disclaimer, éligibilité évaluation, top-N communauté).

Transport / en-têtes

- Caddy : HTTPS automatique, HSTS 2 ans `includeSubDomains` ; `preload` , `X-Content-Type-Options` , `X-Frame-Options: DENY` , `Referrer-Policy` , `Permissions-Policy` (caméra/géoloc/paiement désactivés, micro `self`), CSP, `-Server` . Helmet côté API.

Données / fichiers

- Requêtes 100 % paramétrées ; validation Zod ; validation magic-bytes de l'audio ; déduplication SHA-256 ; URL audio signées HMAC liées à l'utilisateur ; audit échantillonné des accès audio ; gardes anti-traversée sur les suppressions/recoupages ; trigger d'immuabilité de l'identité ; répertoire CSV admin en `0o700` avec purge horaire ; coffre `failed_submissions` .

Anti-abus / intégrité — cf. §11.17 (dispositif complet).

Infrastructure

- Conteneurs non-root ; base sans port publié ; limites mémoire ; healthchecks ; durcissement OS scripté (SSH 2222, `PermitRootLogin no` , `PasswordAuthentication no` , `MaxAuthTries 3` , UFW deny-in, fail2ban 24 h, `unattended-upgrades`) ; pare-feu cloud Hetzner limité à 2222/80/443.

12.2 Éléments manquants — Absent

DOMAINE	MANQUE
Qualité	Tests unitaires/intégration/e2e, CI, scan de dépendances, scan d'image, SBOM
Base de données	Outil de migration versionné, schéma de référence consolidé, numérotation unique
Secrets	<code>.env.example</code> , gestionnaire de secrets, secrets distincts, rotation
Observabilité	Logs structurés, agrégation, corrélation, métriques, tracing, APM, journal d'accès HTTP, uptime externe
Conformité	Politique de confidentialité, base légale, registre, rétention, droit à l'effacement, DPA (OpenAI/ip-api), chiffrement au repos du NNI et de l'audio

DOMAINE	MANQUE
Résilience	Sauvegardes hors site et chiffrées, test de restauration, RTO/RPO, plan de reprise
Auth	MFA admin, verrouillage par compte, politique de mot de passe robuste, CAPTCHA, gestion des sessions par l'utilisateur
Runtime	Store de rate-limit partagé (Redis), durcissement conteneurs (<code>read_only</code> , <code>cap_drop</code> , <code>no-new-privileges</code>), rotation des logs conteneurs
Documentation	Spécification OpenAPI/Swagger, runbook d'incident, matrice des rôles, schéma d'architecture rendu (seul le <code>.excalidraw</code> source est présent)

12.3 Recommandations prioritaires

P0 — À traiter avant tout nouveau déploiement

#	ACTION	EFFORT	FICHIERS CONCERNÉS
P0-1	Supprimer le compte admin par défaut de <code>init.sql</code> ; créer l'admin par script hors dépôt avec mot de passe aléatoire. Vérifier sur la production que le mot de passe a été changé et que <code>ADMIN_BOOTSTRAP</code> n'est pas positionné.	S	<code>backend/sql/init.sql:283-293</code>
P0-2	Générer 4 secrets indépendants (<code>JWT_SECRET</code> , <code>API_KEY_SECRET</code> , <code>AUDIO_URL_SECRET</code> , <code>OTP_PEPPER</code>), les injecter dans le compose, faire échouer le démarrage si l'un manque en production.	S	<code>config.ts</code> , <code>utils/api-key.ts</code> , <code>utils/audio-token.ts</code> , <code>docker-compose.yml</code>
P0-3	Lier le token dataset ZIP à l'utilisateur (inclure <code>userId</code> dans la charge signée), le rendre à usage unique, journaliser chaque téléchargement.	S	<code>utils/dataset-token.ts</code> , <code>routes/users.ts:661</code> , <code>routes/dataset-export.ts</code>
P0-4	Supprimer PgBouncer du compose (il n'est pas utilisé) ou l'activer avec <code>AUTH_TYPE=scram-sha-256</code> ; isoler <code>waha</code> sur un réseau Docker séparé. Aligner <code>DB_MAX_CONNECTIONS × workers</code> sur <code>max_connections</code> .	S	<code>docker-compose.yml</code>
P0-5	Remplacer <code>Math.random()</code> par <code>crypto.randomInt</code> et le SHA-256 nu par un HMAC poivré pour les codes email/reset.	S	<code>routes/auth.ts:729,805</code>
P0-6	Déclarer <code>ipaddr.js</code> dans <code>backend/package.json</code> et passer à <code>npm ci</code> dans les deux Dockerfiles.	S	<code>backend/package.json</code> , <code>Dockerfile</code> , <code>backend/Dockerfile</code>
P0-7	Ajouter un quorum sur <code>POST /phrases/report</code> (N signalements distincts ou revue admin) + plafond par utilisateur/jour.	S	<code>routes/phrases.ts:224-248</code>
P0-8	Committer un <code>.env.example</code> exhaustif (noms uniquement, valeurs factices) couvrant les 30 variables du §8.	S	racine

P1 — Court terme (2 à 6 semaines)

#	ACTION	EFFORT
P1-1	Reconstruire le schéma de référence : <code>pg_dump --schema-only</code> de la production → schéma baseline versionné (récupère notamment <code>votes</code> , <code>votable_phrases</code> , <code>season0_contributors</code> , absents du dépôt) ; introduire un outil de migration (node-pg-migrate, Flyway, Atlas) ; résoudre les 10 doublons de version. Prérequis absolu de toute migration GCP .	L
P1-2	Renforcer l'authentification : mot de passe ≥ 12 caractères, contrôle contre les mots de passe compromis, verrouillage progressif par compte, émission des alertes <code>suspicious_login</code> (le type existe déjà), MFA pour les comptes admin.	M
P1-3	Sauvegardes : chiffrement (<code>age</code> /GPG), copie hors hôte, contrôle d'intégrité, test de restauration documenté , RTO/RPO.	M
P1-4	Remplacer le blocage du quota starter par une quarantaine réversible avec message explicite.	S
P1-5	Durcir les uploads communautaires : magic-bytes, <code>fileFilter</code> , ré-encodage, quota par utilisateur, <code>Content-Disposition</code> pour les types non prévisualisables.	M
P1-6	CSP à nonces , suppression de <code>unsafe-eval</code> , duplication des en-têtes de sécurité sur le bloc <code>api.*</code> du <code>Caddyfile</code> .	M
P1-7	Logs structurés JSON (pino) + identifiant de corrélation + redaction des PII (NNI, WhatsApp, email, IP) + rotation des logs conteneurs (<code>logging</code> driver).	M
P1-8	Store Redis pour <code>express-rate-limit</code> et <code>antiSpeedMiddleware</code> (plafonds réellement globaux) — brique également nécessaire pour toute mise à l'échelle horizontale.	M
P1-9	Purge de <code>audit_log</code> (rétention glissante + archivage) et index adaptés aux requêtes anti-scraping.	S
P1-10	Amorcer les tests : suite d'intégration sur <code>auth</code> , <code>phrases/submit</code> , <code>validate/submit</code> , <code>adminMiddleware</code> , plus un CI exécutant <code>lint</code> + <code>build</code> + tests + <code>npm audit</code> .	L
P1-11	Passer la géo-IP en HTTPS ou base locale ; ne plus faire dépendre le scoring d'un canal non authentifié.	S
P1-12	Corriger <code>deploy/deploy.sh:43</code> (<code>docker compose down</code> contredit la consigne du README).	S

P2 — Moyen terme

#	ACTION
P2-1	Dossier de conformité : politique de confidentialité, base légale du traitement de la voix (biométrie) et du NNI, registre, durées de conservation, endpoint de suppression de compte/export, DPA avec OpenAI, information des tiers de <code>contributor_credits</code> .
P2-2	Chiffrement applicatif du NNI et du numéro WhatsApp (pgcrypto ou chiffrement côté application avec KMS).
P2-3	Spécification OpenAPI générée pour les 225 endpoints + matrice rôles/endpoints.
P2-4	Observabilité : <code>/metrics</code> Prometheus, healthchecks sur tous les services, uptime externe, suivi d'erreurs.

#	ACTION
P2-5	Épingler toutes les images par digest , vérifier la somme de contrôle de <code>yt-dlp</code> , <code>npm prune --omit=dev</code> dans le runner, <code>read_only</code> / <code>cap_drop</code> / <code>no-new-privileges</code> .
P2-6	Supprimer le code mort (<code>daily-cap.ts</code>), les variables inertes (<code>JWT_ACCESS_EXPIRES</code>), et le rôle <code>moderator</code> inutilisé.
P2-7	Remplacer <code>AntiScraping.tsx</code> (blocage clic droit / Ctrl+U — inefficace, dégrade l'accessibilité) par des contrôles serveur exclusivement.

13. Prérequis techniques pour un déploiement ultérieur sur un socle GCP sécurisé

Cette section identifie ce que **le code actuel** exige et ce qui **doit changer** avant toute exécution sur GCP. Chaque item est marqué  **Recommandé** (proposition) ou  **Bloquant** (le code ne fonctionnera pas sans adaptation).

13.1 Contraintes issues du code (à traiter en premier)

CONTRAINTE CONSTATÉE	PREUVE	CONSÉQUENCE GCP
État sur disque local : audio, clips, médias communautaires, coffre d'échecs écrits via <code>fs</code> sur <code>/data/recordings</code>	<code>phrases.ts:488-492</code> , <code>community.ts:113-116</code> , <code>media-ingest.ts</code>	 Bloquant pour Cloud Run (système de fichiers éphémère et non partagé). Choix : (a) réécrire la couche fichiers vers Cloud Storage (effort important : chemins, URL signées, <code>express.static</code> , ZIP, FFmpeg local) ; (b) conserver un système de fichiers POSIX via GKE + Filestore ou une VM avec disque persistant (aucune modification de code)
Caches et compteurs en mémoire : <code>utils/cache.ts</code> (Map), <code>express-rate-limit</code> , <code>antiSpeedMiddleware</code> , cooldowns d'alerte	<code>utils/cache.ts:16</code> , <code>security.ts:133,265</code>	 Toute mise à l'échelle > 1 instance rend les plafonds incohérents → Memorystore for Redis requis
Invalidation de cache par IPC cluster (<code>process.send</code>)	<code>utils/cache.ts:60-75</code> , <code>server.ts:42-46</code>	 Ne fonctionne qu'entre workers d'un même process ; multi-instances = sessions incohérentes (la révocation de session unique ne se propage plus) → Redis Pub/Sub
Workers de fond <code>setInterval</code> (dispatcher notifications, auto-activation, collusion, news bot, purge, reprise d'ingestion)	<code>server.ts:401-747</code>	 Exigent une instance toujours active . Cloud Run : <code>min-instances=1</code> + CPU always allocated ; sinon extraire ces tâches vers Cloud Scheduler + Cloud Run Jobs
Binaires système : <code>ffmpeg</code> , <code>ffprobe</code> , <code>yt-dlp</code> , <code>python3</code>	<code>backend/Dockerfile:16-18</code>	 Conserver un build Docker (buildpacks exclus). Prévoir CPU/mémoire pour le ré-encodage (jobs jusqu'à 1800 s)
Jobs longs (ingestion jusqu'à 30 min)	<code>media-ingest.ts:176</code>	 Dépasse le timeout par défaut de Cloud Run (60 min max configurable) → préférer Cloud Run Jobs ou GKE
<code>NEXT_PUBLIC_API_URL</code> figé au build	<code>Dockerfile:8-9</code>	 Une image frontend par environnement, ou passage à une configuration runtime
72 migrations manuelles + 3 objets DB absents du dépôt	<code>backend/sql/</code> , §4.4.2	 Bloquant : impossible de provisionner Cloud SQL depuis le dépôt. Le schéma de référence doit être extrait de la production (<code>pg_dump --schema-only</code>) avant toute migration (cf. P1-1)
Upload jusqu'à 800 Mo	<code>media.ts:41</code>	 Cloud Run limite la requête à 32 Mo ; le mécanisme de chunks 30 Mo existant (<code>media.ts:162-197</code>) est compatible, l'endpoint <code>/ingest-file</code> non

13.2 Fondation GCP

🔦 Recommandé

DOMAINE	PRÉREQUIS
Organisation	Hiérarchie de dossiers, projets séparés <code>elson-prod</code> / <code>elson-staging</code> , Organization Policies : <code>constraints/sql.restrictPublicIp</code> , <code>constraints/storage.publicAccessPrevention</code> , <code>constraints/compute.requireShieldedVm</code> , <code>constraints/iam.disableServiceAccountKeyCreation</code> , <code>constraints/compute.vmExternalIpAccess</code>
Réseau	VPC dédié, sous-réseaux privés régionaux, Private Service Access pour Cloud SQL, Serverless VPC Access (si Cloud Run), Cloud NAT pour l'egress (OpenAI, ip-api, RSS, YouTube, SMTP), règles de pare-feu en deny-by-default, journaux de flux VPC
Egress maîtrisé	Le backend appelle 6 destinations externes (\$4.5) → allow-list explicite via Cloud NAT + éventuellement Secure Web Proxy
IAM	Un service account dédié par charge de travail (backend, frontend, WAHA, jobs), aucun usage du SA Compute par défaut, principe du moindre privilège, Workload Identity (GKE) ou identités par service (Cloud Run), pas de clés de SA exportées
VPC Service Controls	🔦 Périmètre autour de Cloud SQL, Cloud Storage et Secret Manager pour contenir l'exfiltration du corpus

13.3 Exécution

OPTION	ADÉQUATION AU CODE ACTUEL	PRÉREQUIS
A. GKE Autopilot (recommandé)	✅ La plus proche du modèle actuel : Filestore pour <code>/data/recordings</code> , Deployment backend + StatefulSet WAHA, workers de fond natifs	Cluster privé, Workload Identity, NetworkPolicies (isoler WAHA), PodSecurityStandards <code>restricted</code> , HPA, Filestore (ou GCS via réécriture)
B. Cloud Run	⚠️ Exige les réécritures de §13.1 (GCS + Redis + jobs séparés)	<code>min-instances ≥ 1</code> , CPU always allocated, connecteur VPC, <code>--no-allow-unauthenticated</code> derrière l'ALB, timeout ajusté
C. Compute Engine + Docker Compose (lift & shift)	✅ Zéro modification de code — migration la plus rapide	MIG avec Shielded VM , OS Login, pas d'IP externe, disque persistant chiffré CMEK, Ops Agent, patch management

⚠️ **WAHA** : image tierce détenant une **session WhatsApp persistante** (volume `/app/.sessions`). À traiter comme un composant à privilèges élevés : namespace/projet dédié, service account propre, **egress restreint**, NetworkPolicy interdisant l'accès à Cloud SQL, volume chiffré.

13.4 Données

COMPOSANT	PRÉREQUIS GCP
Cloud SQL for PostgreSQL 17	IP privée uniquement , <code>require_ssl</code> , chiffrement CMEK (Cloud KMS), sauvegardes automatiques + PITR , HA régionale , fenêtre de maintenance, <code>cloudsql.iam_authentication</code> , Query Insights. Remplacer PgBouncer par le Cloud SQL Auth Proxy / connecteur (pooling côté application aligné : <code>DB_MAX_CONNECTIONS × workers ≤ max_connections</code>)

COMPOSANT	PRÉREQUIS GCP
Cloud Storage (audio)	Bucket régional aligné sur la juridiction retenue, uniform bucket-level access , <code>publicAccessPrevention: enforced</code> , CMEK , versioning, règles de cycle de vie (dérivées d'une politique de conservation à écrire), Object Retention Lock pour les archives légales
⚠ Média communautaire	Aujourd'hui public sans authentification . Sur GCS : ne jamais rendre le bucket public ; servir via l'ALB + Cloud CDN avec URL signées
Memorystore for Redis	AUTH activé, chiffrement en transit, IP privée — support des rate-limits, du cache et de l'invalidation multi-instances
Secret Manager	12 secrets à provisionner : <code>DB_PASSWORD</code> , <code>JWT_SECRET</code> , <code>API_KEY_SECRET</code> , <code>AUDIO_URL_SECRET</code> , <code>OTP_PEPPER</code> , <code>SMTP_USER</code> , <code>SMTP_PASS</code> , <code>WAHA_API_KEY</code> , <code>OPENAI_API_KEY</code> , <code>ALERT_EMAILS</code> , <code>API_KEY</code> (legacy), <code>DATASET_UPLOAD_DIR</code> si distinct. Versionnement, rotation planifiée , accès par SA, jamais dans une variable d'environnement en clair d'image
Migration des données	<code>pg_dump</code> → Cloud SQL (ou Database Migration Service) ; <code>gcloud storage rsync</code> du volume <code>recordings</code> ; vérification d'intégrité (nombre de fichiers, sommes de contrôle) ; réconciliation <code>contributions.audio_url</code> ↔ objets GCS

13.5 Périmètre et exposition

💡 Recommandé

COMPOSANT	PRÉREQUIS
Global External Application Load Balancer	Remplace la terminaison TLS de Caddy ; certificats managés ; HTTP/3 ; redirection HTTPS ; HSTS conservé
Cloud Armor	Politique WAF (règles préconfigurées OWASP), rate limiting au bord (complète les limiteurs applicatifs), règles géographiques (la compétition est ciblée Mauritanie/diaspora), protection L7 DDoS, Adaptive Protection
Cloud CDN	Cache des assets statiques et de <code>_landing.html</code> (130 Ko) ; exclude impérativement <code>/api/*</code> , <code>/recordings/*</code> et <code>sw.js</code> (déjà <code>no-store</code> côté Caddy)
Identity-Aware Proxy	💡 Placer les interfaces admin (<code>/admin</code> , <code>/reviewer</code> , <code>/media-studio</code> , <code>/eval-control</code> , <code>/news-bot</code>) derrière IAP : remplace la fragile whitelist IP par une authentification Google + contexte d'accès. Réduit directement les risques \$11.1 et \$11.6
En-têtes de sécurité	Reporter les en-têtes du <code>Caddyfile</code> sur l'ALB (ou conserver Caddy en routeur interne) ; corriger la CSP au passage
Cloudflare	⚠ Décider explicitement du maintien : le code lit <code>CF-Connecting-IP</code> pour la whitelist IP admin et le keying des rate-limits (<code>middleware/auth.ts:224</code> , <code>security.ts:41</code>). En le retirant, il faut adapter le code à <code>X-Forwarded-For</code> tel que produit par l'ALB, sinon la whitelist admin et les plafonds cassent

13.6 Observabilité et exploitation

💡 Recommandé

DOMAINE	PRÉREQUIS
Cloud Logging	Nécessite des logs structurés JSON — le code n'émet que du <code>console.*</code> non structuré (§10.1) → prérequis P1-7. Sinks vers BigQuery pour l'audit, buckets de logs avec rétention

DOMAINE	PRÉREQUIS
Redaction PII	⚠️ Obligatoire avant activation : les logs et emails d'alerte contiennent NNI, WhatsApp, email, IP (\$10.4). Prévoir des règles d'exclusion / Cloud DLP
Cloud Monitoring	Uptime check sur <code>/api/health</code> , alertes (latence, 5xx, saturation du pool DB, profondeur de la file de notifications, remplissage disque/GCS — remplaçant <code>disk-alert.sh</code>), SLO
Error Reporting / Cloud Trace	Prérequis : instrumentation OpenTelemetry (absente)
Cloud Audit Logs	Admin Activity + Data Access sur Cloud SQL, Storage et Secret Manager ; sink immuable

13.7 CI/CD et chaîne d'approvisionnement

💡 Recommandé

ÉTAPE	PRÉREQUIS
Artifact Registry	Dépôts privés <code>elson-backend</code> / <code>elson-frontend</code> , Artifact Analysis (scan de vulnérabilités), politiques de nettoyage
Cloud Build (ou GitHub Actions + WIF)	Pipeline : <code>lint</code> → <code>build</code> → <code>tests</code> (à créer) → <code>npm audit</code> → build image → scan → push → migration DB → déploiement. Fédération d'identité (pas de clé de SA)
Binary Authorization	N'admettre que des images signées et scannées
Reproductibilité	⚠️ Prérequis : <code>npm ci</code> , images épinglées par digest, checksum <code>yt-dlp</code> (\$9.6)
Étape de migration	⚠️ Prérequis : outil de migration (P1-1) — aujourd'hui impossible à automatiser

13.8 Conformité et résidence des données

⚠️ À traiter avant mise en production sur GCP

POINT	EXIGENCE
Région	Choisir explicitement la région Cloud SQL / GCS selon la juridiction applicable aux données mauritaniennes et le RGPD (utilisateurs de la diaspora) ; documenter le choix
Nature des données	Voix (biométrie), NNI (identifiant national), WhatsApp, IP, empreinte de device, décision automatisée de rejet à l'inscription → analyse d'impact (AIPD/DPIA) nécessaire
Sous-traitants	OpenAI (audio + textes envoyés), ip-api.com (IP), Zoho (emails), WhatsApp/Meta via WAHA → DPA et information des personnes
Rétention	Définir et implémenter des durées pour : <code>audit_log</code> , <code>signup_ip</code> / <code>signup_geo</code> , <code>otp_codes</code> , <code>failed_submissions</code> , fichiers audio, sauvegardes
Droits des personnes	Endpoints d'export et de suppression de compte (absents)
Chiffrement	CMEK sur Cloud SQL/GCS + 💡 chiffrement applicatif du NNI

13.9 Dimensionnement

⚠ **À confirmer** — les commentaires du code visent **5 000 utilisateurs simultanés** (`docker-compose.yml:22` , `Caddyfile:1` , `deploy/create-server.sh:14`), aujourd'hui sur un `cx32` (4 vCPU / 8 Go). Aucune mesure de charge réelle n'est présente dans le dépôt. 💡 Établir une **campagne de test de charge** avant de dimensionner Cloud SQL (tier, `max_connections`), Memorystore, et les limites d'autoscaling.

14. Checklist finale — état de préparation au déploiement

14.1 Prérequis fonctionnels

#	CRITÈRE	ÉTAT
1	Fonctionnalités cœur implémentées (contribution, évaluation, vote, relecture, export)	✔ Prêt
2	Interface d'administration complète (22 onglets, ~165 endpoints)	✔ Prêt
3	Internationalisation FR/EN/AR + RTL	✔ Prêt
4	PWA + résilience hors ligne (outbox IndexedDB)	✔ Prêt
5	Dispositif anti-fraude et intégrité de la compétition	✔ Prêt
6	Notifications multicanal (in-app, email, WhatsApp)	✔ Prêt — ⚠ dépend de SMTP_* et WAHA_API_KEY
7	Fonctions IA (transcription, traduction, veille)	⚠ Inactives : OPENAI_API_KEY n'est pas transmise au conteneur (§8.4)

14.2 Prérequis techniques

#	CRITÈRE	ÉTAT
8	Build reproductible (npm ci , images épinglées)	✗ Non prêt — npm install , :latest , yt-dlp non vérifié
9	Schéma de base provisionnable automatiquement	✗ BLOQUANT — 72 migrations manuelles, 10 doublons de version, init.sql incomplet, et 3 objets DB (votes , votable_phrases , season0_contributors) absents du dépôt : un environnement reconstruit depuis le dépôt a une fonctionnalité de vote non fonctionnelle
10	Tests automatisés	✗ Non prêt — aucun test
11	Pipeline CI/CD	✗ Non prêt — absent
12	Configuration documentée (.env.example)	✗ Non prêt — fichier absent alors que le README l'exige
13	Scripts de déploiement cohérents	✗ Non prêt — deploy.sh:43 exécute docker compose down , explicitement interdit par le README
14	Dépendances déclarées correctement	✗ Non prêt — ipaddr.js non déclaré (risque de verrouillage admin)
15	Sauvegardes chiffrées, externalisées, testées	✗ Non prêt — locales, en clair, restauration non testée
16	Observabilité (logs structurés, métriques, alerting)	⚠ Partiel — alertes email + audit trail ✔ ; logs structurés, métriques, tracing ✗
17	Mise à l'échelle horizontale possible	✗ Non prêt — état en mémoire (cache, rate-limits, IPC cluster)

14.3 Prérequis sécurité

#	CRITÈRE	ÉTAT
18	Pas de credentials par défaut	❌ BLOQUANT — admin + mot de passe en clair dans <code>init.sql</code> (§11.1)
19	Secrets cryptographiques indépendants	❌ BLOQUANT — <code>JWT_SECRET</code> gouverne 7 mécanismes (§11.2)
20	Base de données non joignable sans authentification	❌ BLOQUANT — PgBouncer <code>AUTH_TYPE=trust</code> sur le réseau partagé avec WAHA (§11.3)
21	Actif principal (corpus) protégé par un contrôle d'accès lié à l'identité	❌ BLOQUANT — token ZIP non lié à un utilisateur (§11.4)
22	Aléa cryptographique pour tous les secrets à usage unique	❌ Non prêt — <code>Math.random()</code> pour les codes email/reset (§11.5)
23	Politique de mot de passe + protection anti-bruteforce par compte	❌ Non prêt — 6 caractères, aucun verrouillage (§11.6)
24	Intégrité du corpus protégée contre un utilisateur unique	❌ Non prêt — <code>POST /phrases/report</code> sans quorum (§11.7)
25	Aucune injection SQL / commande OS	✅ Prêt — 100 % paramétré, <code>execFile</code> sans shell
26	Protection CSRF	✅ Prêt — 4 couches
27	Sessions et tokens (rotation, révocation, session unique)	✅ Prêt
28	Autorisation admin (rôle en base + whitelist IP fail-closed)	✅ Prêt — 💡 renforcer via IAP/MFA
29	En-têtes de sécurité HTTP	⚠️ Partiel — bons en-têtes, mais CSP <code>unsafe-inline</code> / <code>unsafe-eval</code> et absente du domaine API (§11.10)
30	Validation des uploads	⚠️ Partiel — rigoureuse pour l'audio des contributions, inexistante pour les médias communautaires (§11.9)
31	Conteneurs non-root, base non exposée	✅ Prêt — 💡 ajouter <code>read_only</code> / <code>cap_drop</code>
32	Durcissement OS et pare-feu	✅ Prêt — scripts <code>setup-server.sh</code> / <code>create-server.sh</code>
33	Surveillance des dépendances vulnérables	❌ Non prêt — absente
34	Résistance au déni de service	⚠️ Partiel — limiteurs applicatifs présents mais par worker ; WAF ⚠️ à confirmer (§11.15)

14.4 Prérequis conformité

#	CRITÈRE	ÉTAT
35	Politique de confidentialité / mentions légales	❌ Non prêt — absentes
36	Base légale et consentement (voix = biométrie, NNI)	⚠️ Partiel — mécanisme <code>disclaimer</code> versionné présent mais désactivé par défaut
37	Durées de conservation et purge	❌ Non prêt — seuls les <code>refresh_tokens</code> sont purgés

#	CRITÈRE	ÉTAT
38	Droit à l'effacement / portabilité	✗ Non prêt — aucun endpoint
39	Minimisation des PII dans les logs	✗ Non prêt — NNI/WhatsApp/IP en clair dans les alertes et <code>audit_log</code>
40	Encadrement des sous-traitants (OpenAI, ip-api, Zoho, Meta)	✗ Non prêt — aucun DPA documenté
41	PII de tiers (<code>contributor_credits</code>)	✗ Non prêt — aucune base légale, aucune information des personnes

14.5 Verdict

Résultat global

CATÉGORIE	PRÊT	PARTIEL	NON PRÊT
Fonctionnel (7)	6	1	0
Technique (10)	0	1	9
Sécurité (17)	6	4	7
Conformité (7)	0	1	6
Total (41)	12	7	22

● L'application n'est PAS prête pour un déploiement sur un socle GCP sécurisé

4 points bloquants doivent être levés avant tout déploiement, sur GCP comme ailleurs :

1. **Compte administrateur par défaut avec mot de passe en clair dans le dépôt** (§11.1)
2. `JWT_SECRET` unique gouvernant 7 mécanismes cryptographiques (§11.2)
3. **Base de données joignable sans authentification via PgBouncer** `trust` (§11.3)
4. **Token d'export du corpus non lié à un utilisateur** (§11.4)

2 points bloquants supplémentaires spécifiques à GCP :

5. **Le schéma n'est pas provisionnable depuis le dépôt** — 72 migrations manuelles, 10 numéros en doublon, et surtout **3 objets de base utilisés par le code sans aucune définition versionnée** (`votes` , `votable_phrases` , `season0_contributors`) : un environnement reconstruit à partir du dépôt a une fonctionnalité de vote comparatif cassée (§4.4.1 et §4.4.2)
6. **L'état est local au process** — disque local pour les fichiers, caches et rate-limits en mémoire, invalidation par IPC cluster : incompatible avec Cloud Run et avec toute mise à l'échelle horizontale sans les adaptations du §13.1

Nuance importante

⚠ **L'application est manifestement en exploitation** sur son socle actuel (Hetzner + Docker Compose + Caddy + Cloudflare, domaine `elson.adst.ai`), avec un niveau de maturité **élevé** sur son cœur métier : le dispositif anti-fraude, l'anonymisation, la gestion des sessions, la résilience « aucun travail perdu » et la traçabilité sont supérieurs à ce qu'on observe habituellement sur un projet de cette taille. Les commentaires du code témoignent de plusieurs cycles d'audit de sécurité déjà intégrés (« sec-audit fix #1 » à « #15 », « fix B3 », « E1/E2 », « C1 »).

Le verdict négatif porte donc sur **l'industrialisation** (reproductibilité, tests, migrations, secrets, conformité) et sur **quelques défauts de configuration ponctuels mais graves** — non sur la qualité de la logique applicative.

Chemin le plus court vers un déploiement GCP sécurisé

PHASE	CONTENU	ESTIMATION
Phase 1 — Correctifs bloquants	P0-1 à P0-8 (§12.3) : credentials par défaut, secrets séparés, PgBouncer, token dataset, aléa crypto, dépendance manquante, quorum de signalement, <code>.env.example</code>	1 à 2 semaines
Phase 2 — Industrialisation	Outil de migration + schéma de référence consolidé, CI (lint/build/tests/audit), <code>npm ci</code> , images épinglées, premiers tests d'intégration sur les chemins critiques	3 à 5 semaines
Phase 3 — Adaptation GCP	Décision d'architecture (GKE + Filestore = zéro réécriture ; Cloud Run = réécriture GCS + Redis + jobs), Memorystore pour les rate-limits, logs structurés + redaction PII, Secret Manager, Cloud SQL privé CMEK, ALB + Cloud Armor, IAP sur les interfaces admin	4 à 8 semaines
Phase 4 — Conformité et résilience	Politique de confidentialité, AIPD, durées de conservation + purges, endpoints d'export/suppression, DPA, sauvegardes chiffrées hors site + test de restauration , campagne de charge	3 à 6 semaines

Annexe A — Fichiers analysés

Racine : `README.md`, `package.json`, `package-lock.json`, `next.config.ts`, `tsconfig.json`, `eslint.config.mjs`, `postcss.config.mjs`, `Dockerfile`, `docker-compose.yml`, `Caddyfile`, `.gitignore`, `.dockerignore`, `architecture.excalidraw`

Backend : `backend/package.json`, `backend/tsconfig.json`, `backend/Dockerfile`, `backend/docker-entrypoint.sh`, `backend/src/{server,config,db}.ts`, `backend/src/middleware/{auth,security,validate}.ts`, `backend/src/routes/{auth,phrases,validate,users,community,credits,media,reviews,news,eval-admin,dataset-export}.ts`, `backend/src/services/{audio-processor,media-ingest,email,whatsapp>alert,geoip,ai-news,phrase-pipeline}.ts`, `backend/src/utils/{api-key,audio-token,dataset-token,anon,otp,fraud,audit,cache,text,hassaniya,events,referral,daily-cap}.ts`, `backend/sql/init.sql`, `backend/sql/seed.sql`, `backend/sql/migration_v2..v73_*.sql` (72 fichiers, lecture intégrale de `v4_identity`, `v11_password_reset`, `v16_security_hardening`, `v17_anti_fraud`, `v52_whatsapp_otp` ; inventaire structurel des autres)

Frontend : `src/app/layout.tsx`, `src/app/login/page.tsx`, `src/app/admin/page.tsx`, `src/lib/{api,fingerprint,outbox-db}.ts`, `src/components/{Navigation,AntiScraping,HassaniyaInput}.tsx`, inventaire des 20 routes et 23 composants

Déploiement : `deploy/{create-server,setup-server,deploy,disk-alert}.sh`, `deploy/daily-leaderboard*.mjs`, `deploy/top10_email.html`

Autres : `docs/{evaluation-explained,quality-scoring}.md`, `public/{sw.js,manifest.json,_landing.html}`, `scripts/seed_flores.py`

Annexe B — Ce qui n'a pas pu être vérifié depuis le dépôt

- ⚠ **À confirmer** auprès de l'exploitant :
1. Le mot de passe du compte admin par défaut a-t-il été changé en production ?

2. `ADMIN_BOOTSTRAP` est-il positionné sur le serveur ?
 3. Le contenu réel de `.env` (valeurs et présence de `API_KEY_SECRET` , `AUDIO_URL_SECRET` , `OTP_PEPPER` , `OPENAI_API_KEY`)
 4. Quelles migrations ont effectivement été appliquées, et dans quel ordre
 5. La configuration Cloudflare (WAF, cache, rate limiting) référencée dans les commentaires
 6. L'état réel du durcissement OS (les scripts `deploy/` ont-ils été exécutés intégralement ?)
 7. L'existence de sauvegardes hors site en dehors du volume Docker
 8. Les tests e2e suggérés par le filtre `/_e2e/` (`community.ts:383`) — hors dépôt
 9. Les valeurs actives des ~80 clés de `competition_config` (elles gouvernent le comportement de sécurité : `email_verification_required` , `registration_open` , `leaderboard_anonymous` , `disclaimer_enabled` , `community_admin_only` , `schedule_strategy` ...)
 10. La charge réelle observée face à l'objectif de 5 000 utilisateurs simultanés
-

Document généré par analyse statique du dépôt `elson-main`. Aucune modification, aucun déploiement, aucune exécution de l'application. Aucun secret n'est reproduit.